16-878: Advanced Mechatronics Design
Fall 2025
Carnegie Mellon University

Banana Bawp It!
Final Report

Group 1

Project Group Members
Hopewell Feldmann
Min Seo Kim
Katherene Qi
Arnav Srikanth

Table of Contents

Banana Bawp It! Summary

For our project, we wanted to create a game that would rely on both audio and haptic feedback to make it accessible for vision impaired people. *Banana Bawp It!* (no relation to Hasbro's Bop It) is a fun race against the clock game where the player attempts to complete as many tasks in a randomized order as they can within 30 seconds to score points. These tasks include twisting and pulling handles, stomping on pedals, and shaking the controller.

It first begins in a welcome mode where gameplay is triggered by waving your hand in front of a motion sensor. Then an hourglass is flipped over, beginning the 30 second game timer as the game begins, prompting the player with randomized tasks (*Twist it, Pull it, Shake it, Stomp it*). Each successful task completion awards the player a point and will cause the game to prompt the player with another random task. Each successful task completion will also give players non-visual feedback such as sound and vibration. This allows visually impaired players to enjoy and interact with the game just as easily as others.

1. CAD and Dimension Drawings

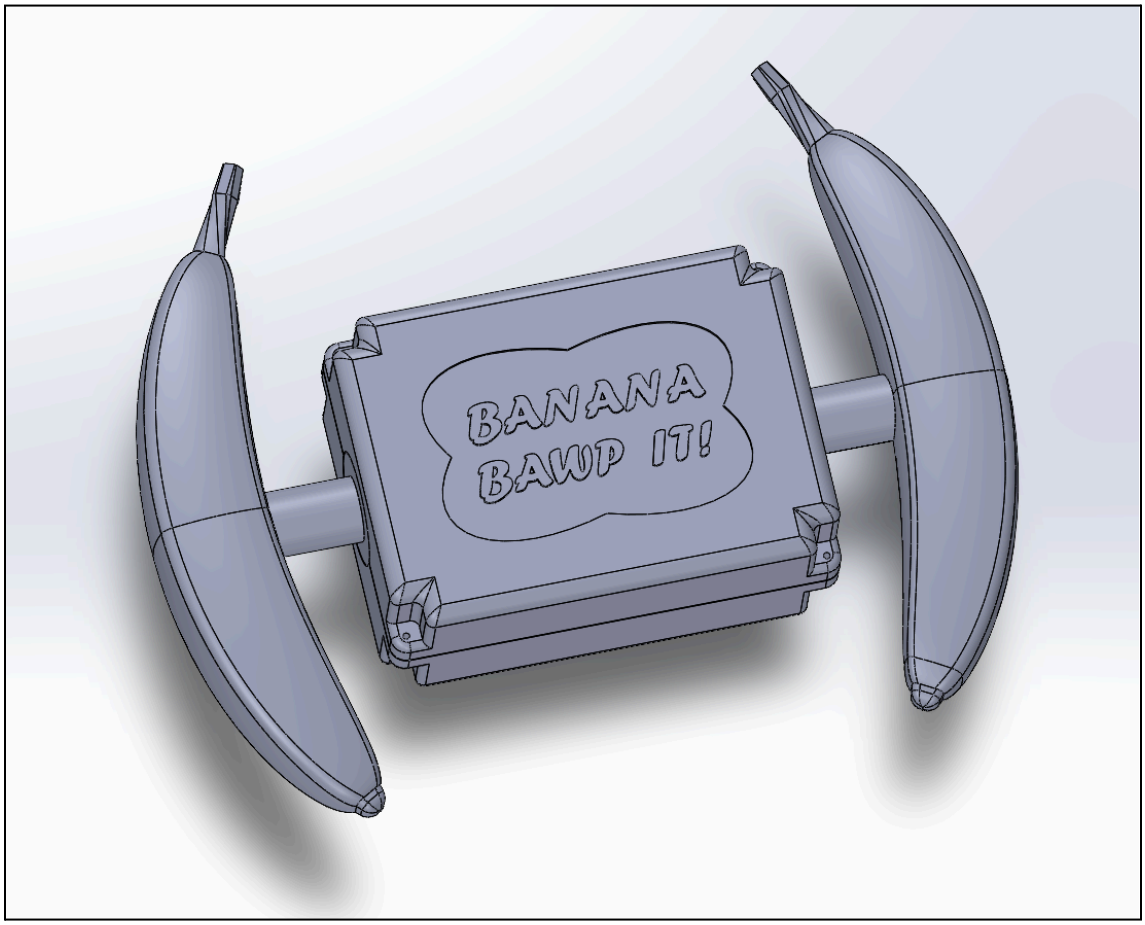

Fig 1: Controller Assembly - Top Down View
Dimensions, LxWxH: 137.8mm x 100mm x 60mm

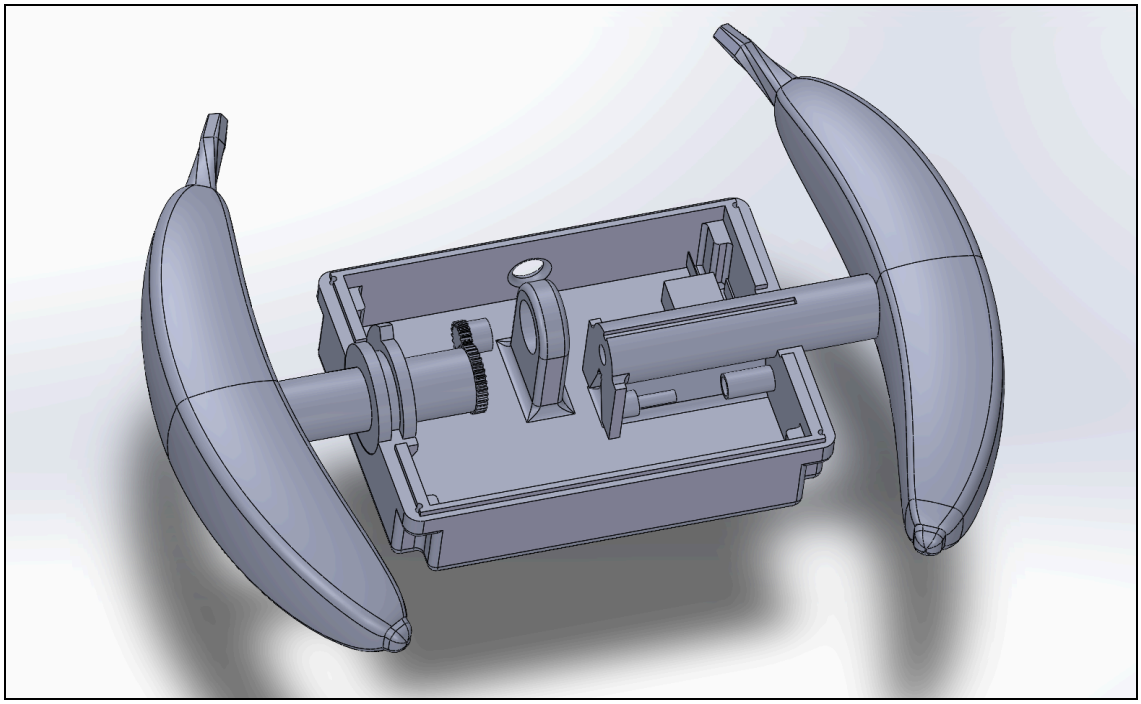

Fig 2: Controller Assembly - Internal View

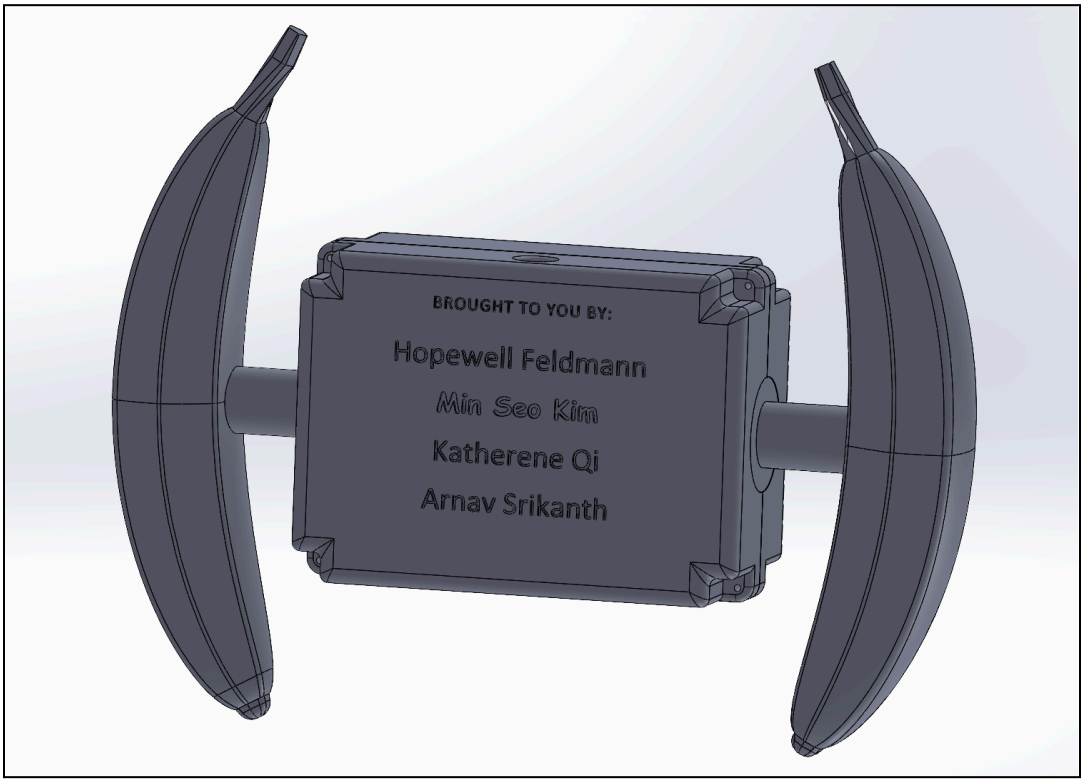

Fig 3: Controller Assembly - Bottom Up View

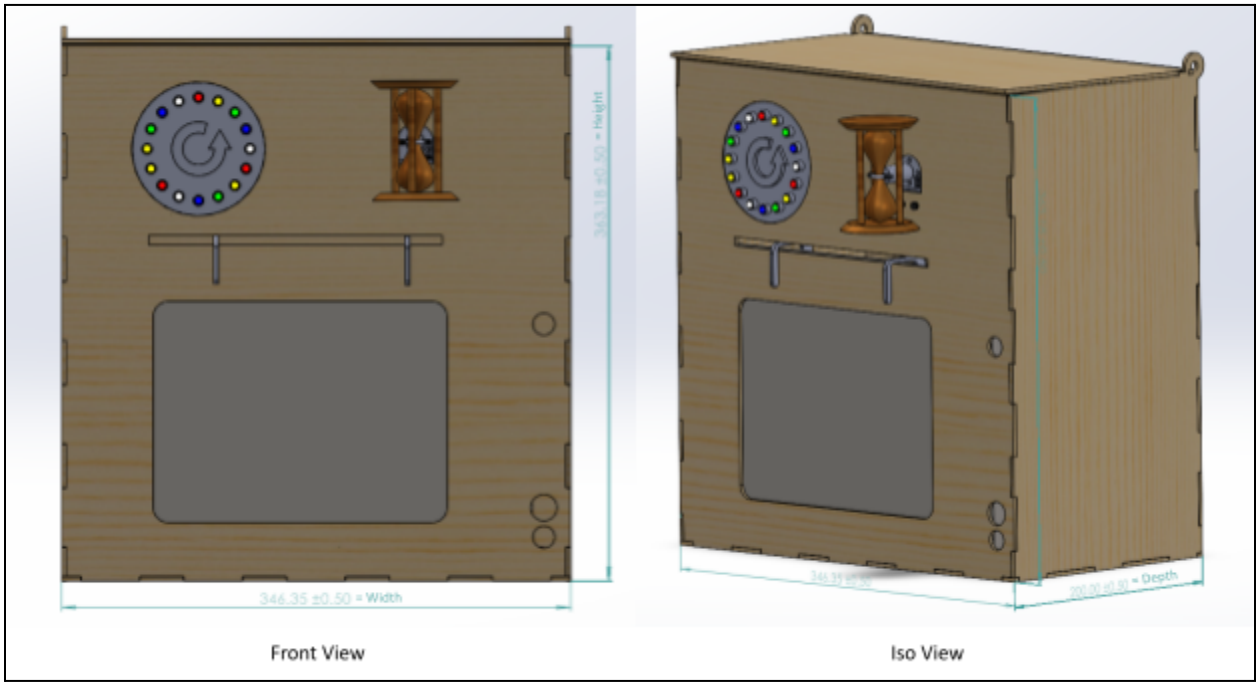

Fig 4: Main Box
Dimensions, WxHxD = 346.35mm x 363.15mm x 200mm

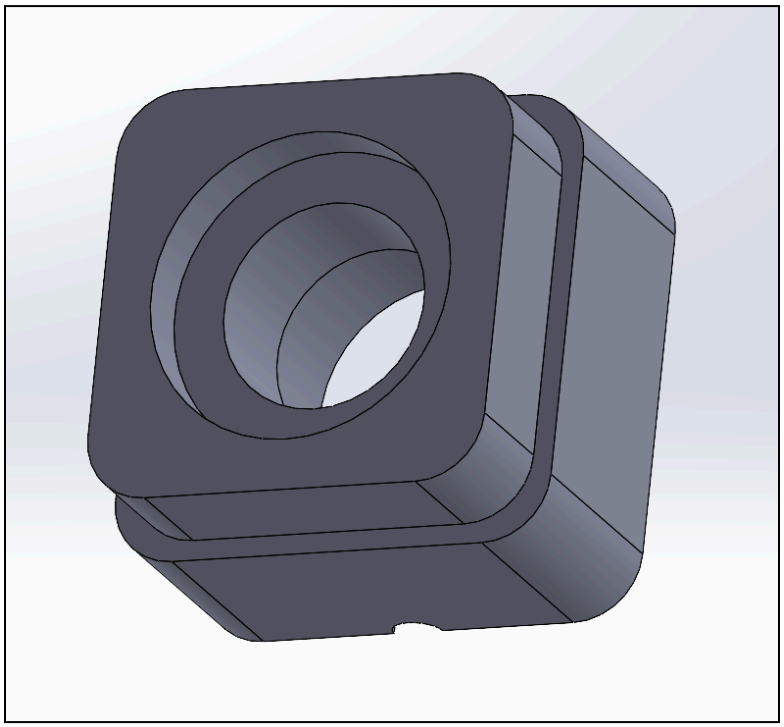

Fig 5: Stomp Button Enclosure
Dimensions, LxWxH: 90mm x 90mm x 66.1mm

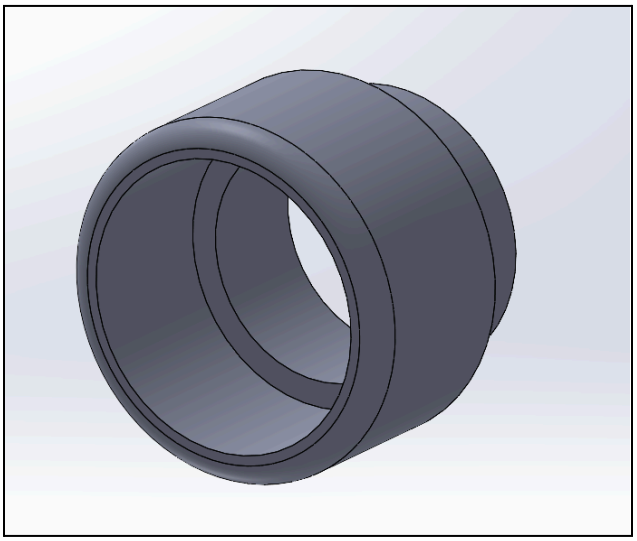

Fig 6: Motion Sensor Mount
Dimensions, RxH: 7.3mm x 14mm

2. Mechatronic System Design

The system consists of several main components: a STM32 Nucleo-F446ZE microcontroller, motor drivers (featuring an open loop and closed loop motor controller), a 16 LED array, limit switches, a motion sensor, and a potentiometer to detect rotation.

Motor Drivers

Both motor controllers are built around an L293B quadruple high-current half-H driver. The open loop controller is designed in such a way that we can turn on and off both motors using a single input from the microcontroller at pin 9, chip enable 2, on the L293B. This works if we have pins 10 and 15 set to low and high respectively. The V_s input at pin 8 is 3.3V as the vibration motors need to run at 3.3V. We elected to use a Pololu 2595 4-channel logic level shifter board to step down the voltage from 5V to 3.3V to avoid potentially damaging the microcontroller by directly pulling current from it. This level shifter was also used to supply 3.3V to other components in the system.

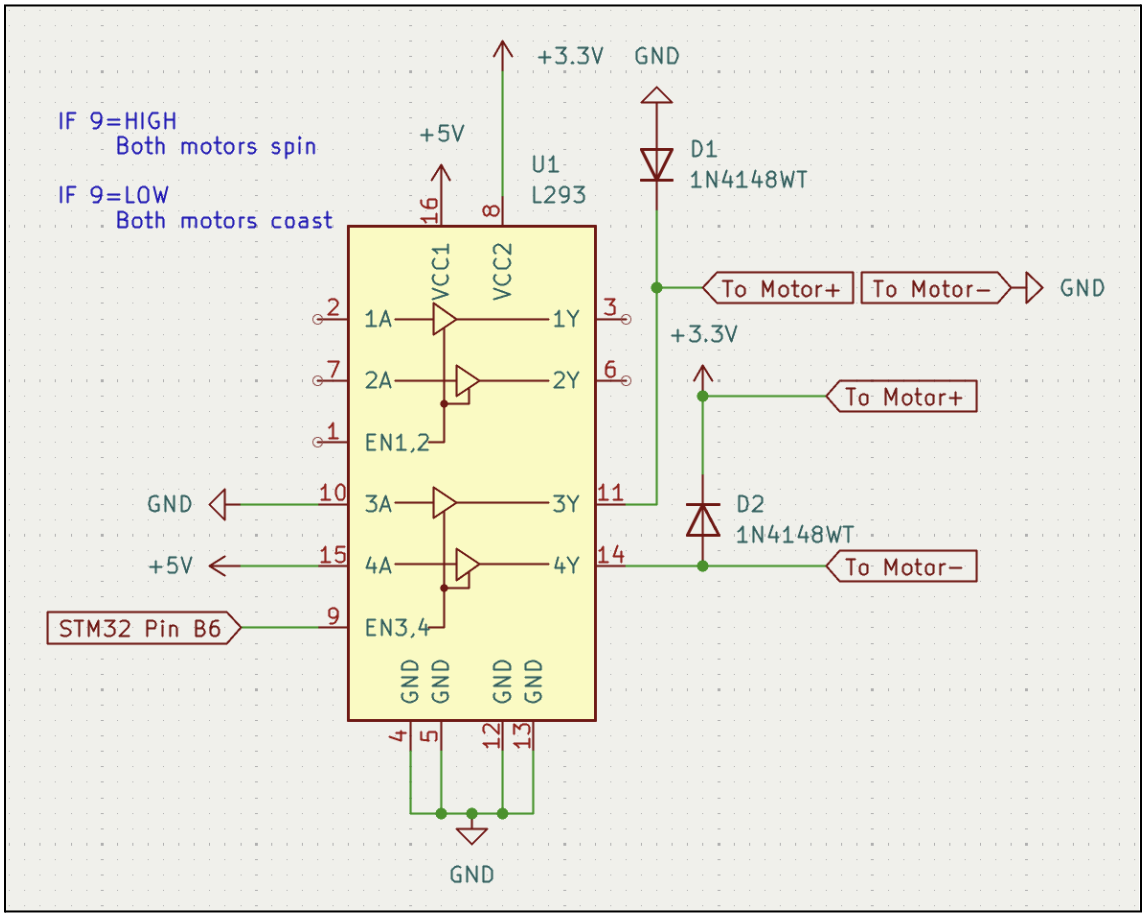

Fig 7. Open loop motor controller for vibration motors.

The open loop motor controller is slightly more complex. Using two inputs from the motor controller instead of one, we are able to achieve bidirectional control over our Pololu 4860 12V DC motor. Bidirectional control is key because we are employing a PID controller to ensure that the hourglass remains vertical during gameplay to achieve accurate timing. Although very similar, there are slight differences to the inputs to the pins on the L293B. Pin 9 is now permanently connected to 5V as we determined that coasting is not required for our application. Pins 10 and 15 serve as logic inputs from the microcontroller to turn the motor either clockwise or anticlockwise. Pin 8, or V_s , is set to 12V to match the motor's 12V operating voltage.

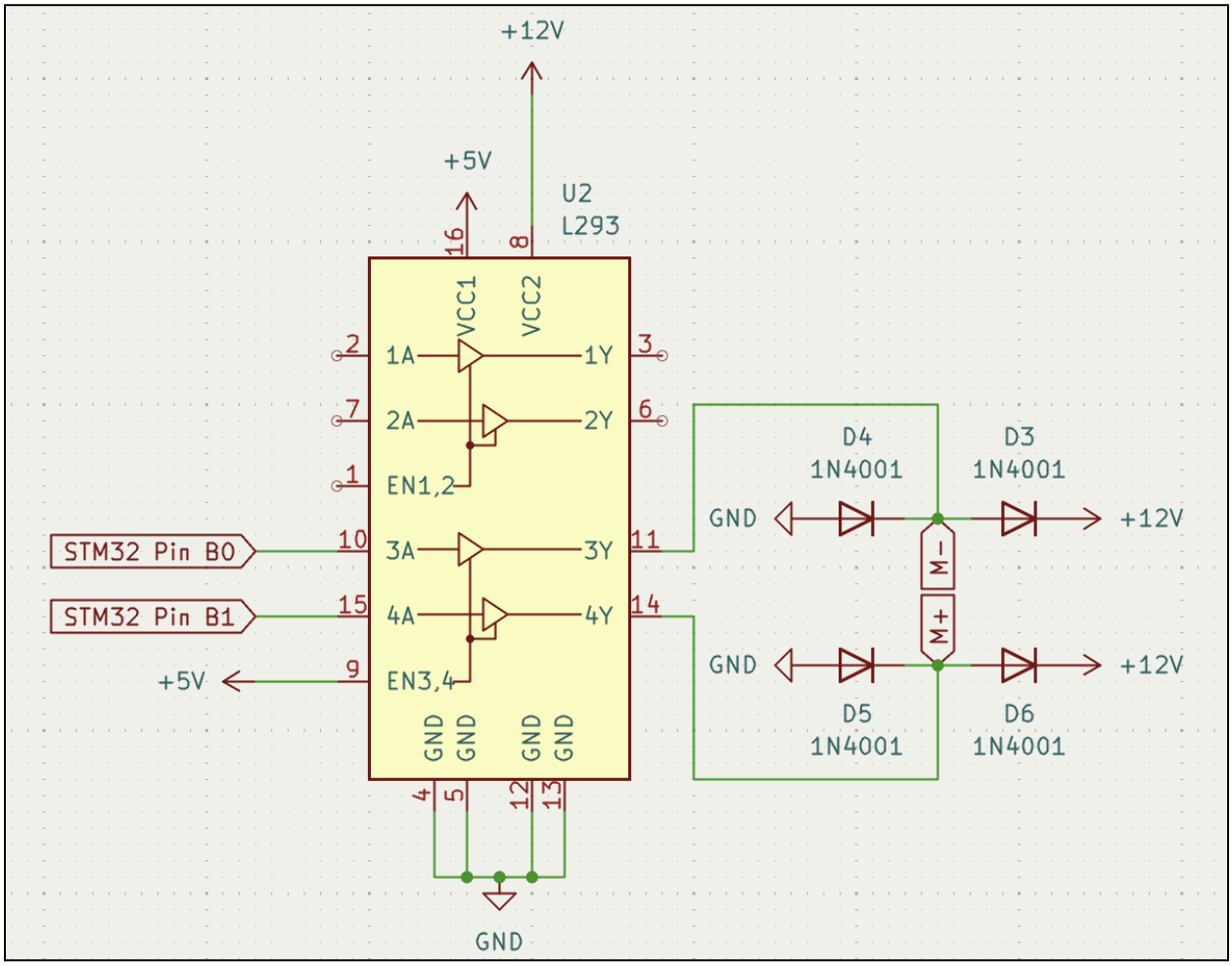

Fig 8. Closed loop motor controller for hourglass motor.

LED Array

One key feature is visual feedback for the potentiometer output. Our solution was an LED array that would light 16 LEDs in a circular shape as the potentiometer value increased. This is accomplished by employing two shift registers, each controlling eight LEDs. As the potentiometer turns, the output voltage value swings between 3.3V and 0V and that indicates to the microcontroller how many lights to turn on. If the potentiometer turns in the wrong direction, LEDs begin to turn off to indicate reversal of progress.

When turning on each individual LED, the shift register gets a bit passed to pin 2, DSB, and the clock pin to trigger a bit shift. In order to turn LEDs off when the user turns the potentiometer backwards (to 0V), we rapidly reset the shift register and shift in the desired amount of bits. This happens so quickly it is indistinguishable to the human eye.

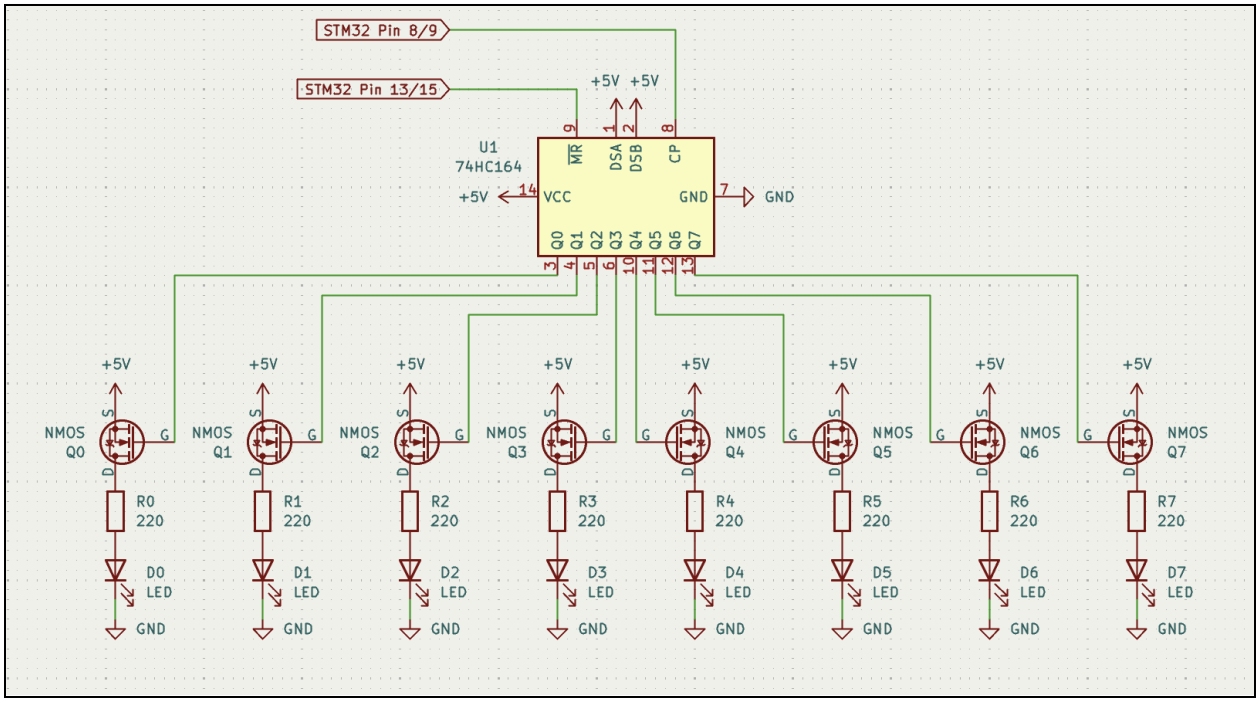

Fig 9. One half of the LED array circuit.

Limit Switches

We incorporated limit switches to check for the completion of three user tasks: pull, stomp, and shake. We use a software debounce for the limit switch to read a rising edge blah blah blah to prevent the microcontroller from accidentally interpreting a single press as multiple rapid presses. For all limit switches, a debounce state machine is used. The goal of the debouncer is to prevent false button presses. More information about how the debouncer works can be found in Section 5.

For the pull task, we designed a tab onto the part of the handle enclosed in the controller box that pressed a limit switch when the handle is pulled out to the maximum length.

For our stomp task, we have a button placed on the floor that triggers a limit switch when stepped on.

To detect the shaking, we originally tried using a limit switch with a weight bouncing against it but found that the minimum activation force required was too large. Thus we devised a solution that used two hex nuts sliding along a vertical rod. Each of these nuts are soldered with wire, and as the controller is shaken, the nuts shake back and forth and repeatedly create rising edges. The microcontroller then reads these edges and once the desired amount of rising edges are detected, the task is flagged as complete.

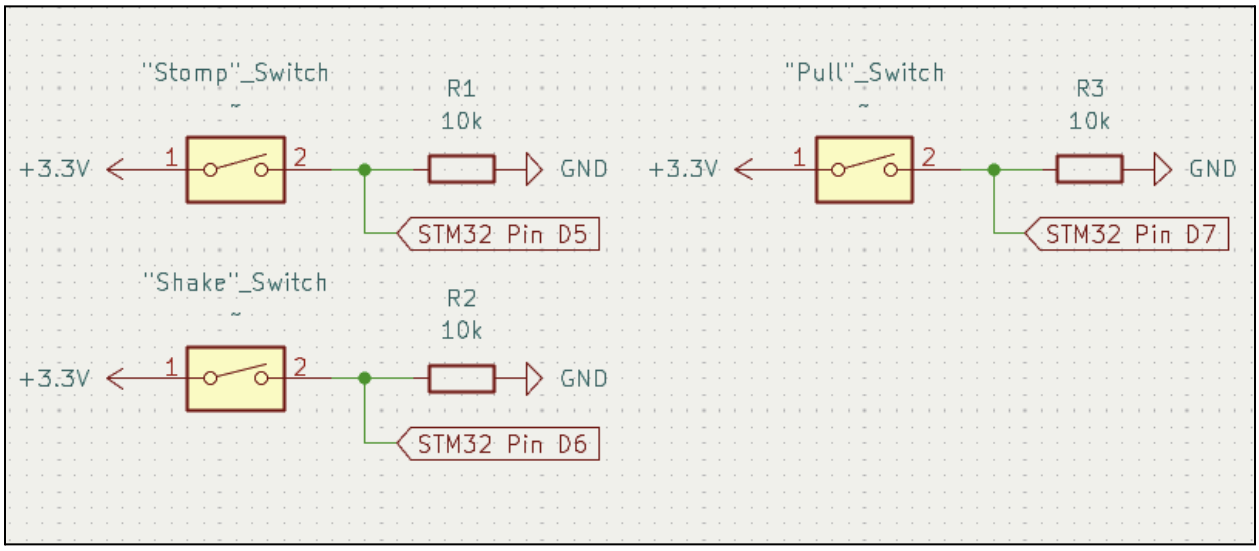

Fig 10. The three limit switch circuits.

Motion Sensor

To detect the hand wave that triggers the start of the game, we used an Adafruit 4871 motion sensor module. This is a fully contained module that employs a passive infrared sensor with all other required components on an attached PCB. It's so convenient that it even holds the high value for 2 seconds when detecting motion and outputs 3.3V so it can be directly connected to the microcontroller pin.

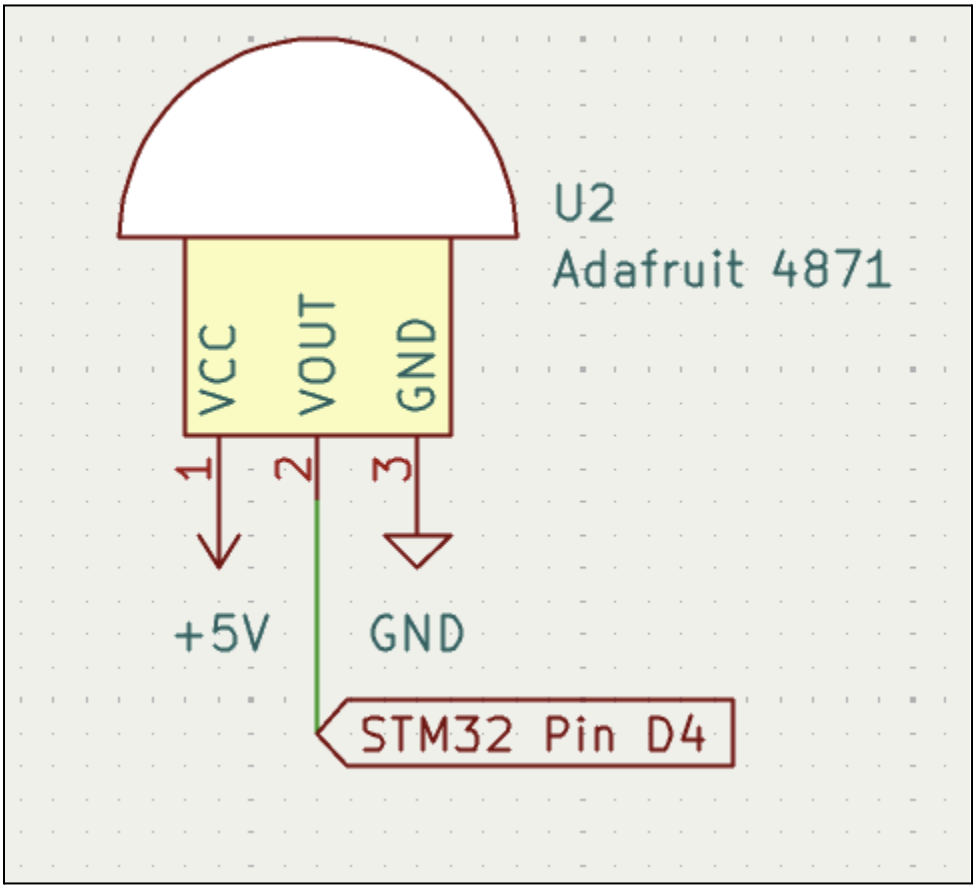

Fig 11. The three limit switch circuits.

Potentiometer

The potentiometer is used to detect an analog signal from the angle one of the handles is twisted. Pin F7 will read the input and then use it to determine the LEDs that are lit up on the main box.

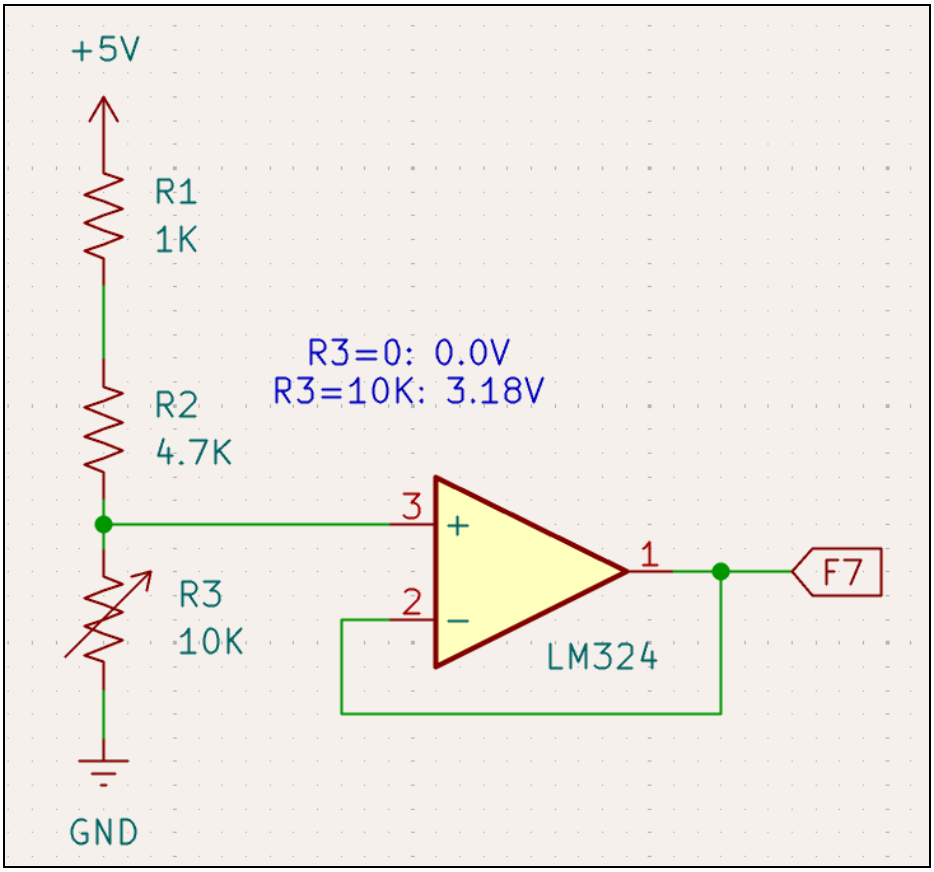

Fig 12. Circuit for the potentiometer.

3. Final Schematics

There are 3 main components of our design:

(1) Main Box
(2) Controller
(3) Stomp Button

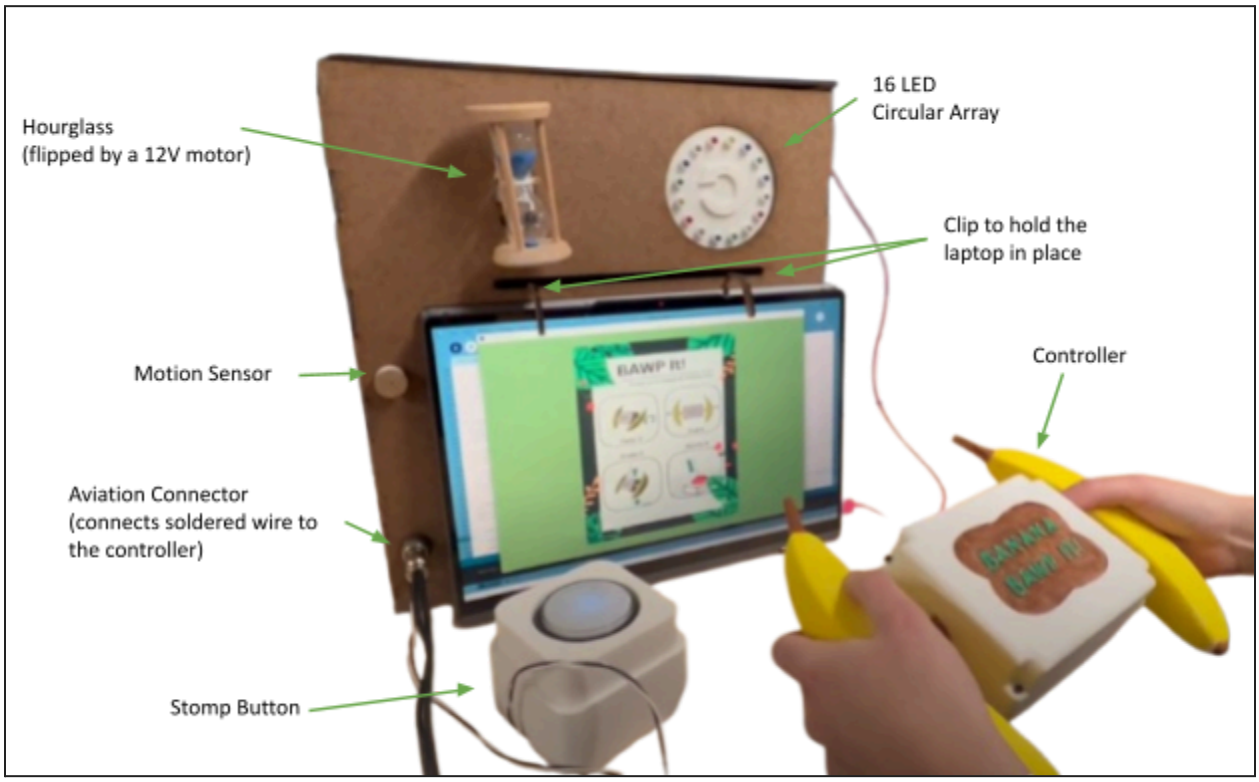

Fig 13: Overview of the system labeled.

Fig 15: Completed Controller - Top Down View

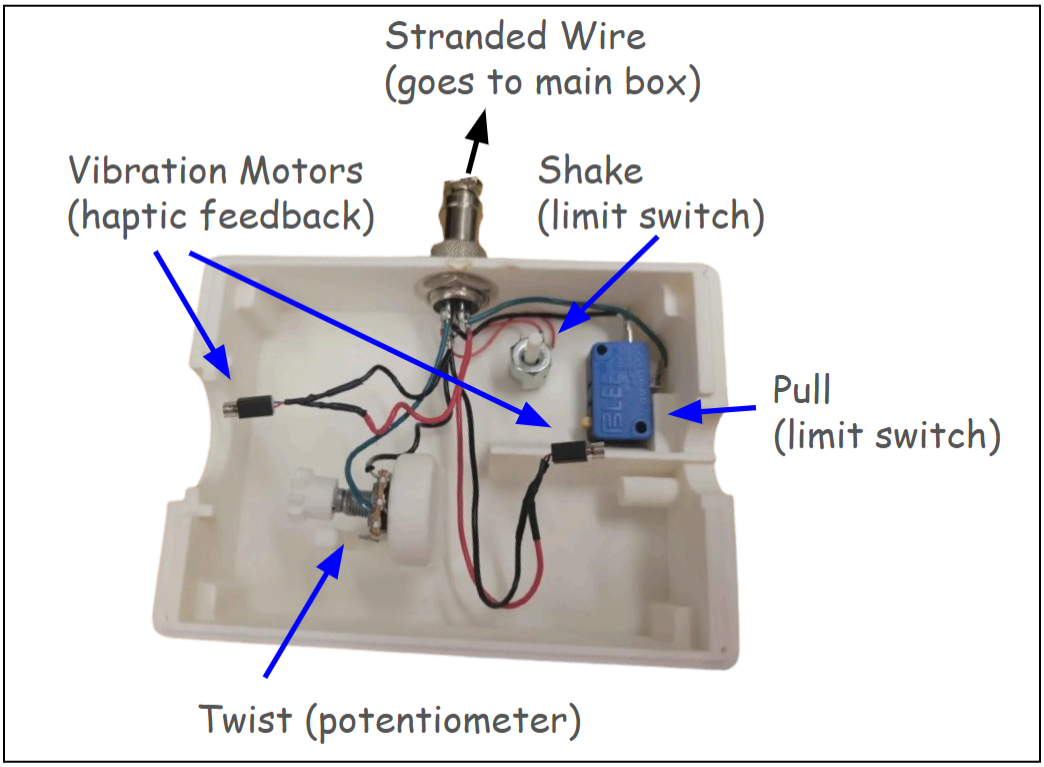

Fig 16: Completed Controller - Internal Electronics

4. Bill of Materials

A full expanded table of our Bill of Materials (BOM) can be found in the following spreadsheet: Total Expense Calculation.

Quantity	Item	Additional Information	Price/item	Vendor
1	200PCS Spring Assortment Kit	Option with “20 sizes”	$7.99	Amazon
1	Sand Timer,30 Seconds	Option for “30 seconds” and the “blue” color	$6.99	Amazon
2	ELEGOO PLA+ Filament 1.75mm White	Option for the “White” color	$14.99	Amazon
3	Motion Sensor	N/A	$7.23	DigiKey
2	Motion Sensor	N/A	$3.95	DigiKey
5	Vibration Motors	N/A	$16.09	DigiKey
2	CONN PLUG HSG 8POS 3.00MM	N/A	$0.52	DigiKey
2	CONN RCPT HSG 8POS 3.00MM	N/A	$0.54	DigiKey
16	CONN PIN 20-24AWG CRIMP TIN	N/A	$0.10	DigiKey
16	CONN SOCKET 20-24AWG CRIMP TIN	N/A	$0.10	DigiKey
1	7 ft of stranded wire	N/A	N/A	Techspark
1	Laser cut Board	N/A	N/A	Course Material
1	Potentiometer	N/A	N/A	Melissa’s Bin
18	Heat Shrink	N/A	N/A	MetaMobility
1	Hot Glue (gun + wax)	N/A	N/A	TechSpark
4	Screws	N/A	N/A	MetaMobility

> 50	Electrical Wires	N/A	N/A	Techspark
4	Protoboards	N/A	N/A	LabKit
N/A	Paint	Yellow, Green, Brown Colors	N/A	Provided by Katherene
6	Connection Dip Sockets	N/A	N/A	From Techspark
2	GX-16 7 Pin Connector	N/A	N/A	MetaMobility

Total Cost: $81.50

5. State Machine Diagrams and Tables

State Diagrams

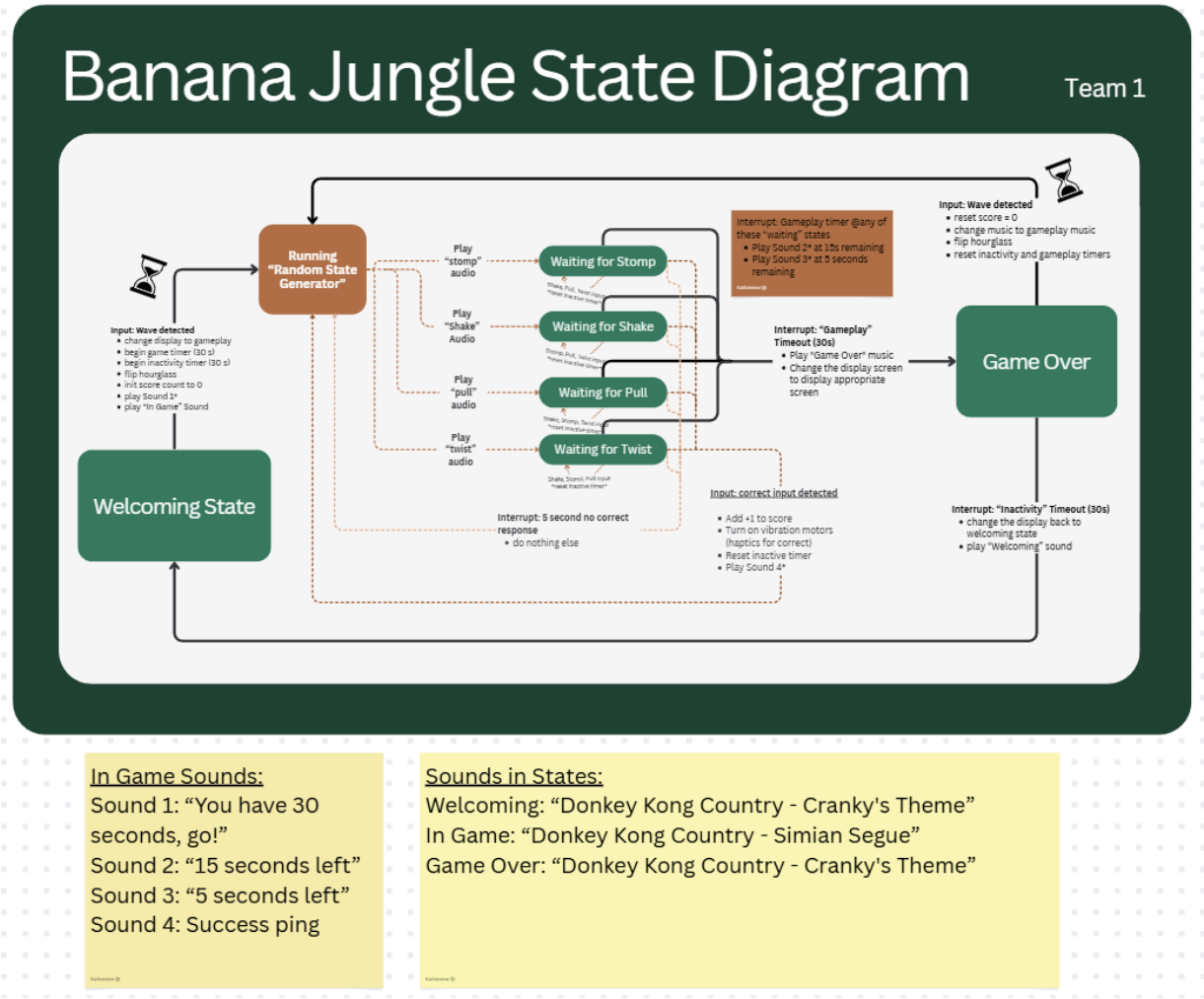

Fig 17: Main State Machine Diagram

One thing to keep in mind is that the "no input timeout" and "game timeout" are both 30 seconds. The input timeout will reset every time an action is registered (stomp, pull, twist, shake). If in the case that the game was started but no one plays, then we will prioritize the "no input timeout" to happen first.

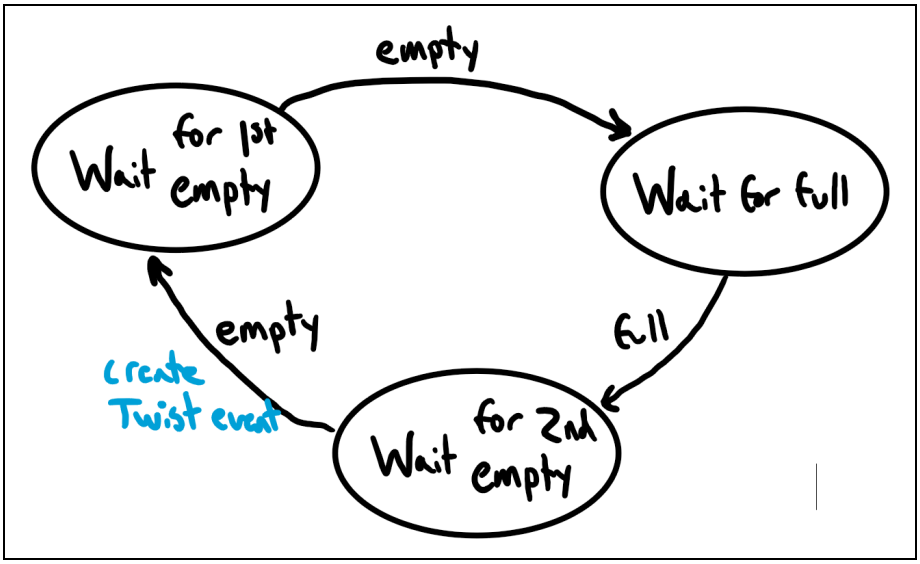

Fig 18. Twist State Machine

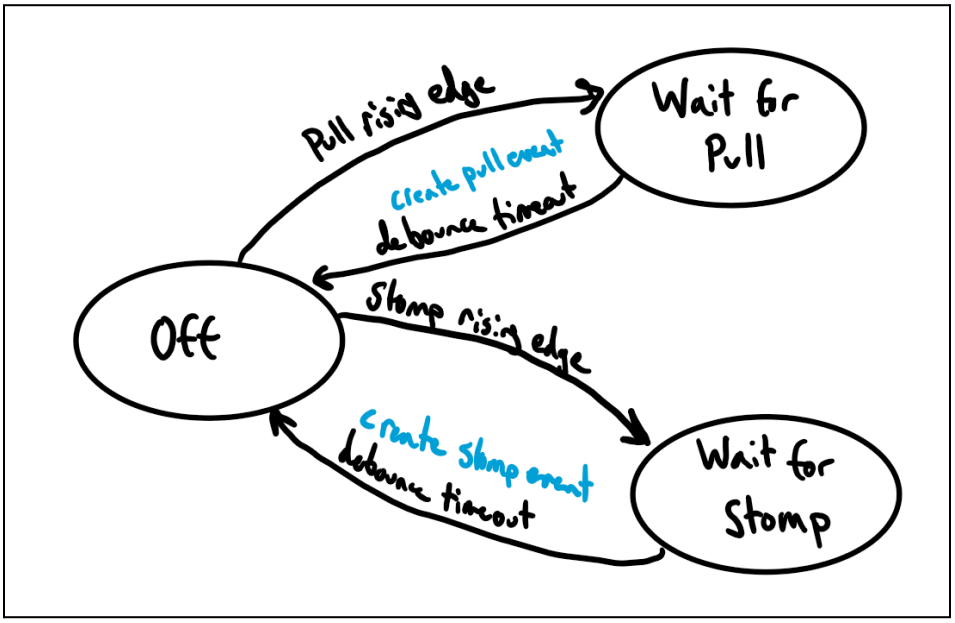

Fig 19: Debounce State Machine

State Tables

The following are the 3 state machine tables that show the logic behind the code. Figure 20 is the main state machine table, where the following figures (twist and debounce) show the supporting state machines that support a smoother gameplay experience. For a larger viewing experience of the main state machine table, see this document: State Machine Table.

State Name	State Description	Event / Condition	Next State	Action during Transition	Notes
Welcoming	- play welcoming music - hourglass should have all the sand at the bottom - if applicable, display corresponding GIF on display screen - display of instructions	Receive a waving input (from the motion sensor) Enough time has passed s.t. the hourglass sand is all on one side	Controller randomizes the next state to go to (one of four options of "waiting for <action>")	- change the music to game music - begin game timer (30 seconds) - flip hourglass - change small screen that displays GIF (if applicable) - init score count to 0 - play the corresponding audio to the next state ("stomp", "shake", "pull", "twist")	* because our gameplay time (30 seconds) is <= inactivity time (30s), we don't have to worry about ensuring that the hourglass has fully reset before starting a new game We would worry about this if gameplay > inactivity time; the "Welcoming" exiting action and "Game Over" restart the game is the only time an hourglass should flip
Waiting for Stomp	- if other actions (shake, pull, twist) are detected, ignore the input - play some idle music ("in play" music)	1.) Waiting for the user input of stomping the button (placed on the floor) 2.) Gameplay timer ends (30s up)	1.) Randomize the next state waiting for a user input 2.) Game over state	1.) Vibrate the vibration motor (signals the action was completed) 2.) Play "end game" music + change the display screen to display appropriate screen (described in "Game Over"'s State Description	
Waiting for Shake	- if other actions (stomp, pull, twist) are detected, ignore the input - play some idle music ("in play" music)	1.) Waiting for the user input of shaking the handle 2.) Gameplay timer ends (30s up)	1.) Randomize the next state waiting for a user input 2.) Game over state	1.) Vibrate the vibration motor (signals the action was completed) 2.) Play "end game" music + change the display screen to display appropriate screen (described in "Game Over"'s State Description	
Waiting for Pull	- if other actions (stomp, shake, twist) are detected, ignore the input - play some idle music ("in play" music)	1.) Waiting for the user input of pulling the handles apart 2.) Gameplay timer ends (30s up)	1.) Randomize the next state waiting for a user input 2.) Game over state	1.) Vibrate the vibration motor (signals the action was completed) 2.) Play "end game" music + change the display screen to display appropriate screen (described in "Game Over"'s State Description	
Waiting for Twist	- if other actions (stomp, shake, pull) are detected, ignore the input - play some idle music ("in play" music)	1.) Waiting for the user input of twisting the handles (aim for max 90 deg twist) 2.) Gameplay timer ends (30s up)	1.) Randomize the next state waiting for a user input 2.) Game over state	1.) Vibrate the vibration motor (signals the action was completed) 2.) Play "end game" music + change the display screen to display appropriate screen (described in "Game Over"'s State Description	
Game Over	- only occurs if the user has played for the full game and did not time out due to inactivity - provides option for user to play again, otherwise it will reset to the welcoming state - displays the player's score from the game (# of correct actions)	1.) Wave (to play again) 2.) Inactivity timer (30s) reached 0s	1.) Randomize the next state waiting for a user input 2.) Returns to welcoming state	1.) Flip the hourglass; reset Gameplay Timer; reset player's score 2.) return to the "Welcoming" State Description	

Fig 20: Main State Machine Table

	Empty	Full
Waiting for 1st Empty	State = waiting for full	Do nothing
Waiting for Full	Do nothing	State = waiting for 2nd empty
Waiting for 2nd Empty	State = waiting for 1st empty Add twist event to list	Do nothing

Fig 21: Twist State Table

In these cases, “empty” refers to the base twisting angle (0°) to the handle. “Full” means twisting the handle to the maximum degrees (~135°). We want to detect that the user has fully twisted the handle and returned the handle to the original position to prevent any shortcuts in detecting future twists (ex. Detecting only that the handle is “full” means a user can leave the handle fully twisted and they’d pass the event very fast).

	Pull rising edge	Stomp rising edge	Timeout
Off	State = waiting for pull Start debounce timer	State = waiting for stomp Start debounce timer	Error (should never happen)
Waiting for pull	Do nothing	Do nothing	State = off If pin high, add pull event to list
Waiting for stomp	Do nothing	Do nothing	State = off If pin high, add stomp event to list

Fig 22: Debounce State Table

6. Header and Code Files

Main.cpp/.h: Contains initializations for pins, interrupts, and timers, and then services the event list in the while loop.
Hardware_stm_gpio.c/.h: Contains functions for initializing pins as analog, inputs, outputs or alternative functions and functions for setting, clearing, or checking pins.
Hardware_stm_adc.c/.h: Code for setting up the ADC and for starting conversions.
Hardware_stm_dma_controller.c/.h: Code for setting up the DMA to work with the ADC and for getting values from the DMA.
Hardware_stm_interruptcontroller.c/.h: Code for enabling interrupts as well as handlers for external interrupts.
Hardware_stm_timer2.c/.h: Initializes channels 1-4 for debouncing and and for controlling motor run times, has functions for starting and stopping channels, and contains the handler for TIM2 interrupts.
Hardware_stm_timer3.c/.h: Initializes TIM3 in PWM mode for running the hourglass motor and has a function for changing the duty cycle.
Hardware_stm_timer4.c/.h: Initializes channels 2-4 as the event timer (seconds until game moves to next event without input), game timer (time to game end), and no action timer (time without user input until game goes to welcoming mode) with start/stop functions for channels and the handler for interrupts.
Motor_control.c/.h: Contains functions for the vibration motors and the hourglass motor including initialization and PID control.
Math.c/.h: Contains a function for producing pseudo random integers.
Led_array.c/.h: Contains functions for controlling the 2 shift registers in order to turn off and on the LEDs in the array based on the analog value from the potentiometer.
Inputs.c/.h: Contains wrapper functions for the GPIO functions for input pins to create better readability.
Events.c/.h: Contains an implementation of a doubly linked list for the event list and the service events function for sorting events to the proper state machine.
State_machine.c/.h: Contains the main state machine that handles direct user events (push, pull, game timeout, etc.).
State_machine_debounce.c/.h: Contains the debounce state machine for the pull and stomp limit switches.
State_machine_twist.c/.h: Contains the twist state machine that determines when a valid twist has happened (LEDs must be all off, then all on, then all off again to generate a twist event).
debug_mort.cpp/.h: Includes debugging print functions as well as functions to print events to serial monitor to communicate with the Processing app.

7. Source Code

A few notes on the code:

- Include statements are excluded for compactness and readability.
- Our PID code worked when the ADC/DMA were not set up but failed when they were. We think this is because the ADC and DMA were interrupting the controller too much or blocking the encoder interrupts. We ran out of time to find a solution (potentially could have changed interrupt priority) and instead went with running the motor for a fixed time. We do show the PID code below, because we think that would have been the correct way to do this.
- We had a recurring bug where sometimes a stomp or pull would not be processed during a stomp or pull event. We think this is because our timers were not set up well and there were too many events firing. It would have been better to have had a global timer instead of so many different timers and channels set up.

Main.cpp

```
int main (void) {
    /* Initializations */
    initLEDs(); // Includes all set up for LED array
    initTIM2ToInterrupt(); // Set up timer 2 to interrupt
    initTIM4ToInterrupt(); // Set up timer 4 to interrupt
    initMotors(); // Init all pins and timers associated with motors
    initAllInputs(); // Init all inputs to interrupt on only rising edges

    /* Service the event list in the loop */
    while(1) {
        serviceEventList();
    }
}
```

Hardware_stm_gpio.c

```
//Port B addresses:
#define PORTB_BASE_ADDRESS ((uint32_t)0x40020400)
#define PORTB_MODER_REGISTER (PORTB_BASE_ADDRESS + 0x00)
#define PORTB_OTYPER_REGISTER (PORTB_BASE_ADDRESS + 0x04)
#define PORTB_OSPEEDR_REGISTER (PORTB_BASE_ADDRESS + 0x08)
#define PORTB_PUPDR_REGISTER (PORTB_BASE_ADDRESS + 0x0C)
#define PORTB_IDR_REGISTER (PORTB_BASE_ADDRESS + 0x10)
#define PORTB_ODR_REGISTER (PORTB_BASE_ADDRESS + 0x14)
```

```
#define PORTB_AFRL_REGISTER (PORTB_BASE_ADDRESS + 0x20)
#define PORTB_AFR2_REGISTER (PORTB_BASE_ADDRESS + 0x24)

//Port F addresses:
#define PORTF_BASE_ADDRESS ((uint32_t)0x40021400)
#define PORTF_MODER_REGISTER (PORTF_BASE_ADDRESS + 0x00)
#define PORTF_OTYPER_REGISTER (PORTF_BASE_ADDRESS + 0x04)
#define PORTF_OSPEEDR_REGISTER (PORTF_BASE_ADDRESS + 0x08)
#define PORTF_PUPDR_REGISTER (PORTF_BASE_ADDRESS + 0x0C)
#define PORTF_IDR_REGISTER (PORTF_BASE_ADDRESS + 0x10)
#define PORTF_ODR_REGISTER (PORTF_BASE_ADDRESS + 0x14)
#define PORTF_LCKR_REGISTER (PORTF_BASE_ADDRESS + 0x1C)
#define PORTF_AFRL_REGISTER (PORTF_BASE_ADDRESS + 0x20)
#define PORTF_AFR2_REGISTER (PORTF_BASE_ADDRESS + 0x24)

// GPIO C addresses: 0x4002 0800 - 0x4002 0BFF
#define PORTC_BASE_ADDRESS ((uint32_t)0x40020800)
#define PORTC_MODER_REGISTER (PORTC_BASE_ADDRESS + 0x00)
#define PORTC_OTYPER_REGISTER (PORTC_BASE_ADDRESS + 0x04)
#define PORTC_OSPEEDR_REGISTER (PORTC_BASE_ADDRESS + 0x08)
#define PORTC_PUPDR_REGISTER (PORTC_BASE_ADDRESS + 0x0C)
#define PORTC_ODR_REGISTER (PORTC_BASE_ADDRESS + 0x14)
#define PORTC_IDR_REGISTER (PORTC_BASE_ADDRESS + 0x10)
#define PORTC_AFRL_REGISTER (PORTC_BASE_ADDRESS + 0x20)

// GPIO D addresses: 0x4002 0C00 - 0x4002 0FFF
#define PORTD_BASE_ADDRESS ((uint32_t) 0x40020C00)
#define PORTD_MODER_REGISTER (PORTD_BASE_ADDRESS + 0x00)
#define PORTD_OTYPER_REGISTER (PORTD_BASE_ADDRESS + 0x04)
#define PORTD_OSPEEDR_REGISTER (PORTD_BASE_ADDRESS + 0x08)
#define PORTD_PUPDR_REGISTER (PORTD_BASE_ADDRESS + 0x0C)
#define PORTD_ODR_REGISTER (PORTD_BASE_ADDRESS + 0x14)
#define PORTD_IDR_REGISTER (PORTD_BASE_ADDRESS + 0x10)
#define PORTD_AFRL_REGISTER (PORTD_BASE_ADDRESS + 0x20)

//flags MODER Register:
#define MODER_CLR ~((uint32_t)0x3) // & to clr
#define MODER_SETOUTPUT (uint32_t) 0x1 // | to set
#define MODER_SETINPUT (uint32_t) 0x0
#define MODER_ANALOG (uint32_t) 0x3
```

```
#define MODER_SHIFT (uint32_t) 0x2
#define MODER_BITS (uint32_t) 0x3
#define MODER_SETALT (uint32_t) 0x2

//flags OTYPER Register:
#define OTYPER_CLR ~((uint32_t)0x1)
#define OTYPER_SETPUSHPULL (uint32_t) 0x0
#define OTYPER_SHIFT (uint32_t) 0x1
#define OTYPER_BITS (uint32_t) 0x1

//flags OSPEEDR Register:
#define OSPEEDR_CLR ~((uint32_t)0x3)
#define OSPEEDR_SETHIGH (uint32_t) 0x3
#define OSPEEDR_SHIFT (uint32_t) 0x2
#define OSPEEDR_BITS (uint32_t) 0x3

//flags PUPDR Register:
#define PUPDR_CLR ~((uint32_t)0x3)
#define PUPDR_SETPULLDOWN (uint32_t) 0x2
#define PUPDR_SETPULLUP (uint32_t) 0x1
#define PUPDR_SHIFT (uint32_t) 0x2
#define PUPDR_BITS (uint32_t) 0x3

//flags ODR Register:
#define ODR_BIT0 (uint32_t)0x1
#define ODR_BITS 0x1
#define ODR_CLR ~((uint32_t)0x1)
#define ODR_SETHI (uint32_t)0x1
#define ODR_SHIFT 0x1

//input data register:
#define READ_IDR_BIT (uint32_t) (0x1)

//flags AFRL Register:
#define AFRL_CLR ~((uint32_t) (0x0F))
#define AFRL_AF2 (uint32_t) 0x2

// B0 is mapped to TIM3 in PWM mode
void initGpioB0AsAF2(void) {
    uint32_t  * reg_pointer;
```

```
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOB, ENABLE);
    /* Set to alternate function mode */
    reg_pointer = (uint32_t*) PORTB_MODER_REGISTER;
    *reg_pointer = *reg_pointer & MODER_CLR;
    *reg_pointer = *reg_pointer | MODER_SETALT;
    /* Set to push pull */
    reg_pointer = (uint32_t*) PORTB_OTYPER_REGISTER;
    *reg_pointer = *reg_pointer & OTYPER_CLR;
    *reg_pointer = *reg_pointer | OTYPER_SETPUSHPULL;
    /* Set to high speed */
    reg_pointer = (uint32_t*) PORTB_OSPEEDR_REGISTER;
    *reg_pointer = *reg_pointer & OSPEEDR_CLR;
    *reg_pointer = *reg_pointer | OSPEEDR_SETHIGH;
    /* Set to floating */
    reg_pointer = (uint32_t*) PORTB_PUPDR_REGISTER;
    *reg_pointer = *reg_pointer & PUPDR_CLR;
    /* Set to alternate function 2*/
    reg_pointer = (uint32_t*) PORTB_AFRL_REGISTER;
    *reg_pointer = *reg_pointer & AFRL_CLR;
    *reg_pointer = *reg_pointer | AFRL_AF2;
}

void initGpioFAsAnalog( uint16_t pin ) {
    uint32_t * reg_pointer;
    /* Enable the AHB1 clock for GPIO F clock */
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOF, ENABLE);
    /* Set at analog */
    reg_pointer = (uint32_t*) PORTF_MODER_REGISTER;
    *reg_pointer = *reg_pointer & ~((MODER_BITS << (MODER_SHIFT * pin)));
    *reg_pointer = *reg_pointer | (MODER_ANALOG << (MODER_SHIFT * pin));
    /* Set as push-pull */
    reg_pointer = (uint32_t*) PORTF_OTYPER_REGISTER;
    *reg_pointer = *reg_pointer & ~((OTYPER_BITS << (OTYPER_SHIFT *
pin)));
    *reg_pointer = *reg_pointer | (OTYPER_SETPUSHPULL << (OTYPER_SHIFT *
pin));
    /* Set as floating */
    reg_pointer = (uint32_t*) PORTF_PUPDR_REGISTER;
    *reg_pointer = *reg_pointer & ~((PUPDR_BITS << (PUPDR_SHIFT * pin)));
}
```

```
void initGpioCAsInput(uint16_t pin) {
    uint32_t  * reg_pointer;
    /* GPIOC Peripheral clock enable */
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOC, ENABLE);
    /* GPIOC as input*/
    reg_pointer = (uint32_t*) PORTC_MODER_REGISTER;
    *reg_pointer = *reg_pointer & ~((MODER_BITS << (MODER_SHIFT * pin)));
    *reg_pointer = *reg_pointer | (MODER_SETINPUT << (MODER_SHIFT * pin));
    /* PUSH-PULL Pin*/
    reg_pointer = (uint32_t*) PORTC_OTYPER_REGISTER;
    *reg_pointer = *reg_pointer & ~((OTYPER_BITS << (OTYPER_SHIFT *
pin)));
    *reg_pointer = *reg_pointer | (OTYPER_SETPUSHPULL << (OTYPER_SHIFT *
pin));
    /* GPIOC pin high speed */
    reg_pointer = (uint32_t*) PORTC_OSPEEDR_REGISTER;
    *reg_pointer = *reg_pointer & ~(OSPEEDR_BITS << (OSPEEDR_SHIFT *
pin));
    *reg_pointer = *reg_pointer | (OSPEEDR_SETHIGH << (OSPEEDR_SHIFT *
pin));
    /* Configure as floating */
    reg_pointer = (uint32_t*) PORTC_PUPDR_REGISTER;
    *reg_pointer = *reg_pointer & ~((PUPDR_BITS << (PUPDR_SHIFT * pin)));
}

void initGpioDAsInput(uint16_t pin) {
    uint32_t  * reg_pointer;
    /* GPIOD Peripheral clock enable */
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOD, ENABLE);
    /* GPIOD as input*/
    reg_pointer = (uint32_t*) PORTD_MODER_REGISTER;
    *reg_pointer = *reg_pointer & ~((MODER_BITS << (MODER_SHIFT * pin)));
    *reg_pointer = *reg_pointer | (MODER_SETINPUT << (MODER_SHIFT * pin));
    /*Push-pull Pin*/
    reg_pointer = (uint32_t*) PORTD_OTYPER_REGISTER;
    *reg_pointer = *reg_pointer & ~((OTYPER_BITS << (OTYPER_SHIFT *
pin)));
    *reg_pointer = *reg_pointer | (OTYPER_SETPUSHPULL << (OTYPER_SHIFT *
pin));
```

```
    /*GPIOD pin high speed */
    reg_pointer = (uint32_t*) PORTD_OSPEEDR_REGISTER;
    *reg_pointer = *reg_pointer & ~(OSPEEDR_BITS << (OSPEEDR_SHIFT *
pin));
    *reg_pointer = *reg_pointer | (OSPEEDR_SETHIGH << (OSPEEDR_SHIFT *
pin));
    /*Configure as floating (must be floating for motion sensor to work
properly) */
    reg_pointer = (uint32_t*) PORTD_PUPDR_REGISTER;
    *reg_pointer = *reg_pointer & ~((PUPDR_BITS << (PUPDR_SHIFT * pin)));
}

void initGpioBAsOutput( uint16_t pin ) {
    uint32_t  * reg_pointer;
    /* GPIOB Peripheral clock enable */
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOB, ENABLE);
    /* GPIOB configured as output */
    reg_pointer = (uint32_t*) PORTB_MODER_REGISTER;
    *reg_pointer = *reg_pointer & ~((MODER_BITS << (MODER_SHIFT * pin)));
    *reg_pointer = *reg_pointer | (MODER_SETOUTPUT << (MODER_SHIFT *
pin));
    /* GPIOB configured as push-pull */
    reg_pointer = (uint32_t*) PORTB_OTYPER_REGISTER;
    *reg_pointer = *reg_pointer & ~((OTYPER_BITS << (OTYPER_SHIFT *
pin)));
    *reg_pointer = *reg_pointer | (OTYPER_SETPUSHPULL << (OTYPER_SHIFT *
pin));
    /* GPIOB configured as high speed (optional) */
    reg_pointer = (uint32_t*) PORTB_OSPEEDR_REGISTER;
    *reg_pointer = *reg_pointer & ~(OSPEEDR_BITS << (OSPEEDR_SHIFT *
pin));
    *reg_pointer = *reg_pointer | (OSPEEDR_SETHIGH << (OSPEEDR_SHIFT *
pin));
    /* GPIOB configured floating */
    reg_pointer = (uint32_t*) PORTB_PUPDR_REGISTER;
    *reg_pointer = *reg_pointer & ~((PUPDR_BITS << (PUPDR_SHIFT * pin)));

    /* GPIOB left low to start out with */
}
```

```
void setGpioB( uint16_t pin) {
    uint32_t  * reg_pointer;
    reg_pointer = (uint32_t*) PORTB_ODR_REGISTER;
    /* Set GpioB high*/
    *reg_pointer = *reg_pointer | (ODR_SETHI << (ODR_SHIFT * pin));
}

void clearGpioB( uint16_t pin) {
    uint32_t  * reg_pointer;
    reg_pointer = (uint32_t*) PORTB_ODR_REGISTER;
    /* Clear GpioB */
    *reg_pointer = *reg_pointer & ~((ODR_BITS << (ODR_SHIFT * pin)));
}

uint32_t checkGpioC(uint16_t pin) {
    uint32_t valueC;
    uint32_t * reg_pointer;
    /* Get and return current value */
    reg_pointer = (uint32_t*) PORTC_IDR_REGISTER;
    valueC = *reg_pointer & (READ_IDR_BIT << pin);
    return valueC;
}

uint32_t checkGpioD(uint16_t pin) {
    uint32_t valueD;
    uint32_t * reg_pointer;
    /* Get and return current value */
    reg_pointer = (uint32_t*) PORTD_IDR_REGISTER;
    valueD = *reg_pointer & (READ_IDR_BIT << pin);
    return valueD;
}

uint32_t checkGpioB(uint16_t pin) {
    uint32_t valueB;
    uint32_t * reg_pointer;
    /* Get and return current value */
    reg_pointer = (uint32_t*) PORTB_IDR_REGISTER;
    valueB = *reg_pointer & (READ_IDR_BIT << pin);
    return valueB;
}
```

Hardware_stm_gpio.h

```
#ifndef __HARDWARE_STM_GPIO_H_
#define __HARDWARE_STM_GPIO_H_

#ifdef __cplusplus
 extern "C" {
#endif

/* Includes
------------------------------------------------------------------*/
#include "main.h"

/*Function
definitions-------------------------------------------------------------*/
void initGpioB0AsAF2(void);
void initGpioFAsAnalog( uint16_t pin );
void initGpioBAsOutput( uint16_t pin );
void initGpioCAsInput(uint16_t pin);
void initGpioDAsInput(uint16_t pin);
void setGpioB( uint16_t pin );
void clearGpioB( uint16_t pin );
uint32_t checkGpioB( uint16_t pin );
uint32_t checkGpioC( uint16_t pin );
uint32_t checkGpioD(uint16_t pin);

#ifdef __cplusplus
}
#endif

#endif
```

Hardware_stm_adc.c

```
// Register addresses
#define ADC_BASE_ADDR (uint32_t) 0x40012000
#define ADC3_BASE_ADDR (ADC_BASE_ADDR + 0x200)
#define ADC_COMMON_BASE_ADDR (ADC_BASE_ADDR + 0x300)
#define ADC_COMMON_CCR_REG (ADC_COMMON_BASE_ADDR + 0x04)
#define ADC3_CR1_REG (ADC3_BASE_ADDR + 0x04)
#define ADC3_CR2_REG  (ADC3_BASE_ADDR + 0x08)
#define ADC3_SQR1 (ADC3_BASE_ADDR + 0x2C)
```

```
#define ADC3_SQR3 (ADC3_BASE_ADDR + 0x34)
#define ADC3_SMPR2 (ADC3_BASE_ADDR + 0x10)
#define ADC3_SR (ADC3_BASE_ADDR + 0x00)
#define ADC3_DR (ADC3_BASE_ADDR + 0x4C)

//Flags
#define CCR_PRE_BITS (((uint32_t) 0x3) << 16)
#define CCR_PRE_4 (((uint32_t) 0x1) << 16)
#define CR2_EOCS (((uint32_t) 0x1) << 10)
#define CR2_CONT (((uint32_t) 0x1) << 1)
#define CR2_DDS (((uint32_t) 0x1) << 9)
#define CR2_DMA (((uint32_t) 0x1) << 8)
#define SQR1_L_3 (((uint32_t) 0x2) << 20)
#define SQR3_5_SQ1 ((uint32_t) 0x5)
#define SQR3_6_SQ2 (((uint32_t) 0x6) << 5)
#define SQR3_7_SQ3 (((uint32_t) 0x7) << 10)
#define SMPR2_5_MAX (((uint32_t) 0x7) << 15)
#define SMPR2_6_MAX (((uint32_t) 0x7) << 18)
#define SMPR2_7_MAX (((uint32_t) 0x7) << 21)
#define CR2_ADON (uint32_t) 0x1
#define CR2_SWSTART (((uint32_t) 0x1) << 30)
#define SR_EOC (uint32_t) 0x2
#define CR1_SCAN_MODE (((uint32_t) 0x1) << 8)
#define CR1_EOCIE (((uint32_t) 0x1) << 5)

void initADC3_5(void) {
    uint32_t * reg_pointer;
    /* Enable the APB2 clock */
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_ADC3, ENABLE);
    /* Init Gpio F7 as analog */
    initGpioFAsAnalog((uint16_t) 7);
    /* Init DMA */
    initDMAForADC3();
    enableDMA();
    /* Init ADC3 channel 5 to continuous conversion */
    // Set up clock prescaler to divide by 4
    reg_pointer = (uint32_t*) ADC_COMMON_CCR_REG;
    *reg_pointer = CCR_PRE_4;
    // Clear status register
    reg_pointer = (uint32_t*) ADC3_SR;
```

```
    *reg_pointer = 0;
    // Set as 12 bit resolution and disable interrupt
    reg_pointer = (uint32_t*) ADC3_CR1_REG;
    *reg_pointer = CR1_SCAN_MODE;
    // Set external trigger disabled, right data alignment, DMA, DDS, EOC
set at end, and enabled continuous conversion
    reg_pointer = (uint32_t*) ADC3_CR2_REG;
    *reg_pointer = CR2_EOCS + CR2_CONT + CR2_DDS + CR2_DMA;
    // Select one conversion
    reg_pointer = (uint32_t*) ADC3_SQR1;
    *reg_pointer = 0;
    // Configure sequence of conversions
    reg_pointer = (uint32_t*) ADC3_SQR3;
    *reg_pointer = SQR3_5_SQ1;
    // Set to 480 cycles
    reg_pointer = (uint32_t*) ADC3_SMPR2;
    *reg_pointer = SMPR2_5_MAX;
    // Turn the ADC on
    reg_pointer = (uint32_t*) ADC3_CR2_REG;
    *reg_pointer = *reg_pointer | CR2_ADON;
}

void startADC3Conversion(void) {
    uint32_t * reg_pointer;
    // Clear status register
    reg_pointer = (uint32_t*) ADC3_SR;
    *reg_pointer = 0;
    // Start converting
    reg_pointer = (uint32_t*) ADC3_CR2_REG;
    *reg_pointer = *reg_pointer | CR2_SWSTART;
}
```

Hardware_stm_adc.h

```
#ifndef __HARDWARE_STM_ADC_H_
#define __HARDWARE_STM_ADC_H_

#ifdef __cplusplus
 extern "C" {
#endif
```

```
/* Includes
------------------------------------------------------------------*/
#include "main.h"

/*Function
definitions-------------------------------------------------------------*/
void initADC3_5(void);
void startADC3Conversion(void);

#ifdef __cplusplus
}
#endif

#endif
```

Hardware_stm_dma_controller.c

```
#define DMA2_BASE_ADDR ((uint32_t) 0x40026400)
#define DMA2_S0CR (DMA2_BASE_ADDR + 0x10)
#define DMA2_S0NDTR (DMA2_BASE_ADDR + 0x14)
#define DMA2_S0PAR (DMA2_BASE_ADDR + 0x18)
#define DMA2_S0M0AR (DMA2_BASE_ADDR + 0x1C)

#define S0CR_CHANNEL2 (((uint32_t) 0x2) << 25)
#define S0CR_MSIZE_HALF (((uint32_t) 0x1) << 13)
#define S0CR_PSIZE_HALF (((uint32_t) 0x1) << 11)
#define S0CR_MINC_INCR (((uint32_t) 0x1) << 10)
#define S0CR_CIRC_ENABLE (((uint32_t) 0x1) << 8)
#define S0CR_DIR_PERTOMEM (uint32_t) 0x0
#define S0CR_STREAM_ENABLE (uint32_t) 0x1

#define ADC_BASE_ADDR (uint32_t) 0x40012000
#define ADC3_BASE_ADDR (uint32_t) (ADC_BASE_ADDR + 0x200)
#define ADC3_DR (uint32_t) (ADC3_BASE_ADDR + 0x4C)

// Array for DMA to store data in (only one value)
uint16_t adcData[1];

void initDMAForADC3(void) {
    uint32_t *reg_pointer;
    /* Enable clock */
```

```
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_DMA2, ENABLE);
    /* Stream 0 as channel 2*/
    reg_pointer = (uint32_t*) DMA2_S0CR;
    *reg_pointer = S0CR_CHANNEL2 + S0CR_MSIZE_HALF + S0CR_PSIZE_HALF +
S0CR_MINC_INCR + S0CR_DIR_PERTOMEM + S0CR_CIRC_ENABLE;
    /* Transfer 1 data register for 1 channesl of ADC */
    reg_pointer = (uint32_t*) DMA2_S0NDTR;
    *reg_pointer = 1;
    /* Transfer from ADC3 data reg */
    reg_pointer = (uint32_t*) DMA2_S0PAR;
    *reg_pointer = ADC3_DR;
    /* Transfer to the variable adcData[] */
    reg_pointer = (uint32_t*) DMA2_S0M0AR;
    *reg_pointer = (uint32_t) (&adcData[0]);
}

// Enable the DMA
void enableDMA(void) {
    uint32_t *reg_pointer;
    reg_pointer = (uint32_t*) DMA2_S0CR;
    *reg_pointer = *reg_pointer | S0CR_STREAM_ENABLE;
}

// Returns the data from the DMA stored in the adcData array
uint16_t getADCData() {
    return adcData[0];
}
```

Hardware_stm_dma_controller.h

```
#ifndef __HARDWARE_STM_DMA_H_
#define __HARDWARE_STM_DMA_H_

#ifdef __cplusplus
 extern "C" {
#endif

/* Includes
------------------------------------------------------------------*/
#include "main.h"
```

```
/*Function
definitions-----------------------------------------------------------------*/
void enableDMA(void);
uint16_t getADCData(void);
void initDMAForADC3(void);

#ifdef __cplusplus
}
#endif

#endif
```

Hardware_stm_interruptcontroller.c

```
// Number of rising edges to generate a shake event
#define SHAKE_LIMIT 40

// NVIC addresses
#define SYSTEM_CONTROL_BASE_ADDRESS (0xE000E000)
#define NVIC_BASE_ADDRESS (SYSTEM_CONTROL_BASE_ADDRESS + 0x100)
#define NVIC_INTERRUPT_SET_ENABLE_REGISTER_0_31 (NVIC_BASE_ADDRESS)

// Interrupt bits
#define EXTI9_5_INTERRUPT_BIT (0x1 << 23)
#define EXTI4_INTERRUPT_BIT (0x1 << 10)
#define TIM3_INTERRUPT_BIT (0x1 << 29)
#define TIM4_INTERRUPT_BIT (0x1 << 30)
#define TIM2_INTERRUPT_BIT (0x1 << 28)
#define ADC_INTERRUPT_BIT (0x1 << 18)

// System config controller
#define SYSCFG_BASE_ADDRESS ((uint32_t)(0x40013800))
#define SYSCFG_EXTICR1 ( SYSCFG_BASE_ADDRESS + 0x08)
#define SYSCFG_EXTICR2 ( SYSCFG_BASE_ADDRESS + 0x0C)
#define SYSCFG_EXTICR3 (SYSCFG_BASE_ADDRESS + 0x10)
#define SYCFG_EXTI_BITS (uint32_t) 0x0F
#define SYCFG_EXTI_SETC (uint32_t) 0x2
#define SYCFG_EXTI_SETD (uint32_t) 0x3
#define SYCFG_EXTI_SHIFT (uint32_t) 0x4
#define SYSCFG_EXTI_6_BITS (uint32_t) 0x0F00
#define SYSCFG_EXTI_6_SETC (uint32_t) 0x200
```

```
#define SYSCFG_EXTI_7_BITS (uint32_t) 0xF000
#define SYSCFG_EXTI_7_SETC (uint32_t) 0x2000
#define SYSCFG_EXTI_8_BITS (uint32_t) 0x0F
#define SYSCFG_EXTI_8_SETC (uint32_t) 0x2
#define SYSCFG_EXTI_9_BITS (uint32_t) 0x0F0
#define SYSCFG_EXTI_9_SETC (uint32_t) 0x20

// External interrupt controller
#define EXTI_BASE_ADDR ((uint32_t)(0x40013C00))
#define EXTI_IMR (EXTI_BASE_ADDR + 0x0) // mask register
#define EXTI_IMR_BIT (uint32_t) 0x1
#define EXTI_IMR_6 (uint32_t) (0x1 << 6)
#define EXTI_IMR_7 (uint32_t) (0x1 << 7)
#define EXTI_IMR_8 (uint32_t) (0x1 << 8)
#define EXTI_IMR_9 (uint32_t) (0x1 << 9)
#define EXTI_RTSR (EXTI_BASE_ADDR + 0x08) // rising edge
#define EXTI_RTSR_BIT (uint32_t) 0x1
#define EXTI_RTSR_6 (uint32_t) (0x1 << 6)
#define EXTI_RTSR_7 (uint32_t) (0x1 << 7)
#define EXTI_RTSR_8 (uint32_t) (0x1 << 8)
#define EXTI_RTSR_9 (uint32_t) (0x1 << 9)
#define EXTI_FTSR (EXTI_BASE_ADDR + 0x0C) // falling edge
#define EXTI_FTSR_7 (uint32_t) (0x1 << 7)
#define EXTI_FTSR_8 (uint32_t) (0x1 << 8)
#define EXTI_FTSR_9 (uint32_t) (0x1 << 9)
#define EXTI_PR (EXTI_BASE_ADDR + 0x14)
#define EXTI_PR_BIT (uint32_t) 0x1
#define EXTI_PR_4 (uint32_t) (0x1 << 4)
#define EXTI_PR_5 (uint32_t) (0x1 << 5)
#define EXTI_PR_6 (uint32_t) (0x1 << 6)
#define EXTI_PR_7 (uint32_t) (0x1 << 7)
#define EXTI_PR_8 (uint32_t) (0x1 << 8)
#define EXTI_PR_9 (uint32_t) (0x1 << 9)

// Enable the NVIC for TIM4
void enableNVIC_Timer4(void) {
    uint32_t * reg_pointer;
    reg_pointer = (uint32_t*) NVIC_INTERRUPT_SET_ENABLE_REGISTER_0_31;
    *reg_pointer = TIM4_INTERRUPT_BIT;
}
```

```
// Enable the NVIC for TIM2
void enableNVIC_Timer2(void) {
    uint32_t * reg_pointer;
    reg_pointer = (uint32_t*) NVIC_INTERRUPT_SET_ENABLE_REGISTER_0_31;
    *reg_pointer = TIM2_INTERRUPT_BIT;
}

// Set pin between 4 and 7 inclusive to interrupt on a rising edge
void enableEXTIOnPortD_4_7(uint16_t pin) {
    uint32_t* reg_pointer;
    /* Enable SYSCFG clock */
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG, ENABLE);
    /* Map external interrupt to C pin# */
    reg_pointer = (uint32_t *) SYSCFG_EXTICR2;
    // - 4 is offset for CR2 reg (CR2 has pins 4 to 7 and CR1 has pins 0
to 3)
    *reg_pointer = *reg_pointer & ~(SYCFG_EXTI_BITS << ((pin - 4) *
SYCFG_EXTI_SHIFT));
    *reg_pointer = *reg_pointer | (SYCFG_EXTI_SETD << ((pin - 4) *
SYCFG_EXTI_SHIFT));
    /* Unmask EXTI in EXTI_IMR register */
    reg_pointer = (uint32_t *) EXTI_IMR;
    *reg_pointer = *reg_pointer | (EXTI_IMR_BIT << pin);
    /* Set to trigger on rising edge */
    reg_pointer = (uint32_t *) EXTI_RTSR;
    *reg_pointer = *reg_pointer | (EXTI_RTSR_BIT << pin);
    /* Set the NVIC to respond to EXTI4 or to EXTI9_5 */
    reg_pointer = (uint32_t *) NVIC_INTERRUPT_SET_ENABLE_REGISTER_0_31;
    if (pin == 4) {
        *reg_pointer = EXTI4_INTERRUPT_BIT;
    } else {
        *reg_pointer = EXTI9_5_INTERRUPT_BIT;
    }
}

// Enable external interrupt on C9 for both rising and falling edges
void enableEXTI9OnPortC(void) {
    uint32_t* reg_pointer;
    /* Enable SYSCFG clock */
```

```
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG, ENABLE);
    /* Map external interrupt to C7 */
    reg_pointer = (uint32_t *) SYSCFG_EXTICR3;
    *reg_pointer = *reg_pointer & ~SYSCFG_EXTI_9_BITS;
    *reg_pointer = *reg_pointer | SYSCFG_EXTI_9_SETC;
    /* Unmask EXTI7 in EXTI_IMR register */
    reg_pointer = (uint32_t *) EXTI_IMR;
    *reg_pointer = *reg_pointer | EXTI_IMR_9;
    /* Set to trigger on rising edge */
    reg_pointer = (uint32_t *) EXTI_RTSR;
    *reg_pointer = *reg_pointer | EXTI_RTSR_9;
    /* Set to trigger on falling edge */
    reg_pointer = (uint32_t *) EXTI_FTSR;
    *reg_pointer = *reg_pointer | EXTI_FTSR_9;
    /* Set the NVIC to respond to EXTI9_5 */
    reg_pointer = (uint32_t *) NVIC_INTERRUPT_SET_ENABLE_REGISTER_0_31;
    *reg_pointer = EXTI9_5_INTERRUPT_BIT;
}

// Enable external interrupt on C8 for both rising and falling edges
void enableEXTI8OnPortC(void) {
    uint32_t* reg_pointer;
    /* Enable SYSCFG clock */
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG, ENABLE);
    /* Map external interrupt to C8 */
    reg_pointer = (uint32_t *) SYSCFG_EXTICR3;
    *reg_pointer = *reg_pointer & ~SYSCFG_EXTI_8_BITS;
    *reg_pointer = *reg_pointer | SYSCFG_EXTI_8_SETC;
    /* Unmask EXTI8 in EXTI_IMR register */
    reg_pointer = (uint32_t *) EXTI_IMR;
    *reg_pointer = *reg_pointer | EXTI_IMR_8;
    /* Set to trigger on rising edge */
    reg_pointer = (uint32_t *) EXTI_RTSR;
    *reg_pointer = *reg_pointer | EXTI_RTSR_8;
    /* Set to trigger on falling edge */
    reg_pointer = (uint32_t *) EXTI_FTSR;
    *reg_pointer = *reg_pointer | EXTI_FTSR_8;
    /* Set the NVIC to respond to EXTI9_5 */
    reg_pointer = (uint32_t *) NVIC_INTERRUPT_SET_ENABLE_REGISTER_0_31;
    *reg_pointer = EXTI9_5_INTERRUPT_BIT;
```

```
}

// Enable the NVIC for ADC interrupt
void enableNVIC_ADC(void) {
    uint32_t *reg_pointer;
    reg_pointer = (uint32_t*) NVIC_INTERRUPT_SET_ENABLE_REGISTER_0_31;
    *reg_pointer = ADC_INTERRUPT_BIT;
}

// Unmask an interrupt in the EXTI_IMR register
void restartEXTI(uint16_t pin) {
    uint32_t* reg_pointer;
    // Re-enable the interrupt
    reg_pointer = (uint32_t *) EXTI_IMR;
    *reg_pointer = *reg_pointer | (EXTI_IMR_BIT << pin);
}

// Mask an interrupt in the EXTI_IMR register
void stopEXTI(uint16_t pin) {
    uint32_t* reg_pointer;
    // Stop flag from being set again
    reg_pointer = (uint32_t *) EXTI_IMR;
    *reg_pointer = *reg_pointer & ~(EXTI_IMR_BIT << pin);
}

// Total count managed by C8 and C9 interrupts (encoder count)
static float count = 0;
// Returns the current count
float getCount(void) {
    return count;
}
// Sets the count to 0 (eg. when motor restarted)
void resetCount(void) {
    count = 0;
}

// D4 is motion sensor pin
// From testing we determined that the motion sensor does not need
debouncing
void EXTI4_IRQHandler(void) {
```

```
    uint32_t * reg_pointer;
    reg_pointer = (uint32_t *) EXTI_PR;
    // Motion sensor rising edge detected
    if ((*reg_pointer & EXTI_PR_4)>0) {
        // Clear the interrupt
        *reg_pointer = EXTI_PR_4;
        // Add a wave event to the event list
        addEvent(WAVE);
    }
}

// Shake count is the current number of rising edges detected by the shake
interrupt
uint16_t shakeCount = 0;
// When a new shake is started, the count is reset
void resetShake(void) {
    shakeCount = 0;
}

void EXTI9_5_IRQHandler(void) {
    uint32_t * reg_pointer;
    reg_pointer = (uint32_t *) EXTI_PR;

    // Stomp rising edge detected
    if ((*reg_pointer & EXTI_PR_5) > 0 ) {
        // Clear the interrupt
        *reg_pointer = EXTI_PR_5;
        // Stop the interrupt
        stopEXTI(5);
        // Add the event to the list
        addEvent(STOMP_RISING_EDGE);
    }
    // Shaking rising edge
    if ((*reg_pointer & EXTI_PR_6) > 0) {
        // Clear the interrupt
        *reg_pointer = EXTI_PR_6;
        // Count the rising edge
        shakeCount = shakeCount + 1;
        // If we have reached the limit, add the event
        if(shakeCount == SHAKE_LIMIT) {
```

```
            shakeCount = 0;
            addEvent(SHAKE);
        }
    }
    // Rising edge from pull detected
    if ((*reg_pointer & EXTI_PR_7) > 0) {
        // Clear the interrupt
        *reg_pointer = EXTI_PR_7;
        // Stop the interrupt
        stopEXTI(7);
        // Add rising edge to the event list
        addEvent(PULL_RISING_EDGE);
    }
    // Encoder A rising or falling fired
    if ((*reg_pointer & EXTI_PR_8)>0) {
        // Clear the interrupt
        *reg_pointer = EXTI_PR_8;
        // Rising edge
        if(checkGpioC(8) > 0) {
            // B is high, direction is ccw
            if(checkGpioC(9) > 0) {
                count = count + 1;
            } else { // B is low, direction is cw
                count = count - 1;
            }
        } else { // Falling edge
            // B is high, direction is cw
            if(checkGpioC(9) > 0) {
                count = count - 1;
            } else { // B is low, direction is ccw
                count = count + 1;
            }
        }
    }
    // Encoder B rising or falling fired
    if ((*reg_pointer & EXTI_PR_9)>0) {
        // Clear the interrupt
        *reg_pointer = EXTI_PR_9;
        // Rising edge
        if(checkGpioC(9) > 0) {
```

```
            // A is high, direction is cw
            if(checkGpioC(8) > 0) {
                count = count - 1;
            } else { // A is low, direction is ccw
                count = count + 1;
            }
        } else { // Falling edge
            // A is high, direction is ccw
            if(checkGpioC(8) > 0) {
                count = count + 1;
            } else { // A is low, direction is cw
                count = count - 1;
            }
        }
    }
}
```

Hardware_stm_interruptcontroller.h

```
#ifndef __HARDWARE_STM_INTERRUPTCONTROLLER_H_
#define __HARDWARE_STM_INTERRUPTCONTROLLER_H_

#ifdef __cplusplus
 extern "C" {
#endif

/* Includes
------------------------------------------------------------------*/
#include "main.h"

/*Function
definitions-------------------------------------------------------------*/
void enableNVIC_ADC(void);
void enableNVIC_Timer4(void);
void enableNVIC_Timer2(void);
void enableEXTI9OnPortC(void);
void enableEXTI8OnPortC(void);
void enableEXTIOnPortD_4_7(uint16_t pin);
void resetCount(void);
float getCount(void);
void resetShake(void);
```

```
void restartEXTI(uint16_t pin);
void stopEXTI(uint16_t pin);

#ifdef __cplusplus
}
#endif

#endif
```

Hardware_stm_timer2.c

```
#define TIM2_BASE_ADDR (uint32_t) 0x40000000
#define TIM2_CR1_REG TIM2_BASE_ADDR
#define TIM2_SR_REG (TIM2_BASE_ADDR + 0x10)
#define TIM2_CAPTURE_COMPARE_MODE_1_REG (TIM2_BASE_ADDR + 0x18)
#define TIM2_CAPTURE_COMPARE_MODE_2_REG (TIM2_BASE_ADDR + 0x1C)
#define TIM2_PRESCALE_REG (TIM2_BASE_ADDR + 0x28)
#define TIM2_AUTORELOAD_REG (TIM2_BASE_ADDR + 0x2C)
#define TIM2_COMP4_REG (TIM2_BASE_ADDR + 0x40)
#define TIM2_COMP3_REG (TIM2_BASE_ADDR +  0x3C)
#define TIM2_COMP2_REG (TIM2_BASE_ADDR + 0x38)
#define TIM2_COMP1_REG (TIM2_BASE_ADDR + 0x34)
#define TIM2_CAPTURE_COMP_ENABLE_REG (TIM2_BASE_ADDR + 0x20)
#define TIM2_INTERRUPT_ENABLE_REG (TIM2_BASE_ADDR + 0x0C)

// Flags for CR1
#define COUNTER_ENABLE (uint16_t) 0x01

// Flags for TIM4_CCMR registers
#define TIM_OC4PE (0x1 << 11)
#define TIM_OC3PE (0x1 << 3)
#define TIM_OC2PE (0x1 << 11)
#define TIM_OC1PE (0x1 << 3)

// Flags for CCER
#define CCER_CC4E (0x1 << 12)
#define CCER_CC3E (0x1 << 8)
#define CCER_CC2E (0x1 << 4)
#define CCER_CC1E (0x1)

// Flags for DIER
```

```
#define CH1_CC_INTERRUPT_ENABLE (uint16_t)(0x1 << 1)
#define CH2_CC_INTERRUPT_ENABLE (uint16_t)(0x1 << 2)
#define CH3_CC_INTERRUPT_ENABLE (uint16_t)(0x1 << 3)
#define CH4_CC_INTERRUPT_ENABLE (uint16_t)(0x1 << 4)
#define UPDATE_INTERRUPT_ENABLE 0x1

// Flags for SR
#define CH1_CC1IF (0x1 << 1)
#define CH2_CC2IF (0x1 << 2)
#define CH3_CC3IF (0x1 << 3)
#define CH4_CC4IF (0x1 << 4)
#define TIM_UIF 0x1

// Inits all 4 channels to the same prescale and auto reload value
void initTIM2ToInterrupt(void) {
    uint16_t * reg_pointer;
    // Values tuned for limit switch debounce
    uint16_t prescale = 4;
    uint16_t auto_reload = 0x13;

    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);
    enableNVIC_Timer2();
    /* Clear update event flag */
     reg_pointer = (uint16_t *) TIM2_SR_REG;
    *reg_pointer = 0;
    /* Upload prescale value */
    reg_pointer = (uint16_t*) TIM2_PRESCALE_REG;
    *reg_pointer = prescale;
    /* Set period value */
    reg_pointer = (uint16_t*) TIM2_AUTORELOAD_REG;
    *reg_pointer = auto_reload;
    /* Set value to compare to (sets duty cycle) */
    reg_pointer = (uint16_t *) TIM2_COMP4_REG;
    *reg_pointer = auto_reload/2;
    reg_pointer = (uint16_t *) TIM2_COMP3_REG;
    *reg_pointer = auto_reload/2;
    reg_pointer = (uint16_t *) TIM2_COMP2_REG;
    *reg_pointer = auto_reload/2;
    reg_pointer = (uint16_t *) TIM2_COMP1_REG;
    *reg_pointer = auto_reload/2;
```

```
    /* Enable preload reg */
    reg_pointer = (uint16_t*) TIM2_CAPTURE_COMPARE_MODE_2_REG;
    *reg_pointer = *reg_pointer | TIM_OC4PE | TIM_OC3PE;
    reg_pointer = (uint16_t*) TIM2_CAPTURE_COMPARE_MODE_1_REG;
    *reg_pointer = *reg_pointer | TIM_OC2PE | TIM_OC1PE;
    /* Enable TIM2 channels 1-4 */
    reg_pointer = (uint16_t *) TIM2_CAPTURE_COMP_ENABLE_REG;
    *reg_pointer = *reg_pointer | CCER_CC4E | CCER_CC3E | CCER_CC2E |
CCER_CC1E;
    /* enable timer subsystem */
     reg_pointer = (uint16_t *) TIM2_CR1_REG;
    *reg_pointer = *reg_pointer | COUNTER_ENABLE;
    /* Channels only enabled in a start() function */
}

// Channel 4 is the stomp debounce timer
void startStompDebounce(void) {
    uint16_t * reg_pointer;
    reg_pointer = (uint16_t *) TIM2_INTERRUPT_ENABLE_REG;
    *reg_pointer = *reg_pointer | CH4_CC_INTERRUPT_ENABLE;
}

void stopStompDebounce(void) {
    uint16_t * reg_pointer;
    reg_pointer = (uint16_t *) TIM2_INTERRUPT_ENABLE_REG;
    *reg_pointer = *reg_pointer & (~CH4_CC_INTERRUPT_ENABLE);
}

// Channel 1 is the pull debounce timer
void startPullDebounce(void) {
    uint16_t * reg_pointer;
    reg_pointer = (uint16_t *) TIM2_INTERRUPT_ENABLE_REG;
    *reg_pointer = *reg_pointer | CH1_CC_INTERRUPT_ENABLE;
}

void stopPullDebounce(void) {
    uint16_t * reg_pointer;
    reg_pointer = (uint16_t *) TIM2_INTERRUPT_ENABLE_REG;
    *reg_pointer = *reg_pointer & (~CH1_CC_INTERRUPT_ENABLE);
}
```

```
// Channel 3 is for vibration motor run time
void startVibrationTimer(void) {
    uint16_t * reg_pointer;
    reg_pointer = (uint16_t *) TIM2_INTERRUPT_ENABLE_REG;
    *reg_pointer = *reg_pointer | CH3_CC_INTERRUPT_ENABLE;
}

void stopVibrationTimer(void) {
    uint16_t * reg_pointer;
    reg_pointer = (uint16_t *) TIM2_INTERRUPT_ENABLE_REG;
    *reg_pointer = *reg_pointer & (~CH3_CC_INTERRUPT_ENABLE);
}

// Channel 2 is the timer for hourglass motor run time
void startMotorTimer(void) {
    uint16_t * reg_pointer;
    reg_pointer = (uint16_t *) TIM2_INTERRUPT_ENABLE_REG;
    *reg_pointer = *reg_pointer | CH2_CC_INTERRUPT_ENABLE;
}

void stopMotorTimer(void) {
    uint16_t * reg_pointer;
    reg_pointer = (uint16_t *) TIM2_INTERRUPT_ENABLE_REG;
    *reg_pointer = *reg_pointer & (~CH2_CC_INTERRUPT_ENABLE);
}

void TIM2_IRQHandler(void) {
    uint16_t * reg_pointer_sr;
    uint16_t * reg_pointer_dier;
    reg_pointer_sr = (uint16_t *) TIM2_SR_REG;
    reg_pointer_dier = (uint16_t *) TIM2_INTERRUPT_ENABLE_REG;
    // Counts for each channel
    static int motor_count = 0;
    static int vibration_count = 0;
    static int debounce_pull = 0;
    static int debounce_stomp = 0;
    // Pull debounce timer fired
    if (( (*reg_pointer_sr & CH1_CC1IF) > 0) && ( (*reg_pointer_dier &
CH1_CC_INTERRUPT_ENABLE) > 0)) {
```

```
        // Clear interrupt
        *reg_pointer_sr = ~((uint16_t)CH1_CC1IF);
        debounce_pull = debounce_pull + 1; // increment count
        // If we have reached desired count, reset, stop timer, and add
event
        if (debounce_pull == 4) {
            debounce_pull = 0;
            stopPullDebounce();
            addEvent(PULL_TIMEOUT);
        }
    }
    // Hourglass motor timer fired
    if (( (*reg_pointer_sr & CH2_CC2IF) > 0) && ( (*reg_pointer_dier &
CH2_CC_INTERRUPT_ENABLE) > 0)) {
        // Clear interrupt
        *reg_pointer_sr = ~((uint16_t)CH2_CC2IF);
        motor_count = motor_count + 1;
        if (motor_count == 5) {
            motor_count = 0;
            setPWMDutyCycle(0); // Turn off motor
            stopMotorTimer(); // Stop timer
        }
    }
    // Stomp debounce timer fired
    if (( (*reg_pointer_sr & CH4_CC4IF) > 0) && ( (*reg_pointer_dier &
CH4_CC_INTERRUPT_ENABLE) > 0)) {
        // Clear interrupt
        *reg_pointer_sr = ~((uint16_t)CH4_CC4IF);
        debounce_stomp = debounce_stomp + 1;
        // If desired count reached, add event
        if (debounce_stomp == 4) {
            debounce_stomp = 0;
            stopStompDebounce();
            addEvent(STOMP_TIMEOUT);
        }
    }
    // Vibration motor timer fired
    if (( (*reg_pointer_sr & CH3_CC3IF) > 0) && ( (*reg_pointer_dier &
CH3_CC_INTERRUPT_ENABLE) > 0)) {
        // Clear interrupt
```

```
        *reg_pointer_sr = ~((uint16_t)CH3_CC3IF);
        vibration_count = vibration_count + 1;
        if (vibration_count == 5) {
            stopVibrationMotors(); // Turn off the vibration motors
            vibration_count = 0;
            stopVibrationTimer(); // Turn off the vibration timer
        }
    }
    // Check if overflow event fired
    if (( (*reg_pointer_sr & TIM_UIF) > 0) && ( (*reg_pointer_dier &
UPDATE_INTERRUPT_ENABLE) > 0)) {
        // Clear interrupt
        *reg_pointer_sr = ~((uint16_t) TIM_UIF);
    }
    // This line did seem to help with some bugs, but not sure it is
correct
    *reg_pointer_sr = 0; // Clear any remaining flags
}
```

Hardware_stm_timer2.h

```
#ifndef __HARDWARE_STM_TIMER2_H_
#define __HARDWARE_STM_TIMER2_H_

#ifdef __cplusplus
 extern "C" {
#endif

/* Includes
------------------------------------------------------------------*/
#include "main.h"

/*Function
definitions-------------------------------------------------------------*/
void initTIM2ToInterrupt(void);
void startPullDebounce(void);
void stopPullDebounce(void);
void startStompDebounce(void);
void stopStompDebounce(void);
void startVibrationTimer(void);
void stopVibrationTimer(void);
```

```
void startMotorTimer(void);
void stopMotorTimer(void);

#ifdef __cplusplus
}
#endif

#endif
```

Hardware_stm_timer3.c

```
#define TIM3_BASE_ADDR (uint32_t) 0x40000400
#define TIM3_CR1_REG TIM3_BASE_ADDR
#define TIM3_SR_REG (TIM3_BASE_ADDR + 0x10)
#define TIM3_CAPTURE_COMPARE_MODE_2_REG (TIM3_BASE_ADDR + 0x1C)
#define TIM3_PRESCALE_REG (TIM3_BASE_ADDR + 0x28)
#define TIM3_AUTORELOAD_REG (TIM3_BASE_ADDR + 0x2C)
#define TIM3_COMP3_REG (TIM3_BASE_ADDR + 0x3C)
#define TIM3_CAPTURE_COMP_ENABLE_REG (TIM3_BASE_ADDR + 0x20)
#define TIM3_CAPTURE_COMP_MODE_1_REG (TIM3_BASE_ADDR + 0x18)
#define TIM3_CAPTURE_COMP_1_REG (TIM3_BASE_ADDR + 0x34)
#define TIM3_INTERRUPT_ENABLE_REG (TIM3_BASE_ADDR + 0x0C)

// Flags for TIM3_CCMR registers
#define TIM_CCMR13_OC1M_0 (0b00010000)
#define TIM_CCMR13_OC1M_1 (0b00100000)
#define TIM_CCMR13_OC1M_2 (0b01000000)
#define TIM_CCMR13_OCPE (0b00001000)
#define TIM_CCMR13_OUTPUT (0x00)
#define TIM_CC1S_INPUT_TL1 (0x01)
#define TIM_OC3M_PWMmode (((uint16_t)(0x6)) << 4)
#define TIM_OC3PE (0x08)
// Flags for CCER
#define CCER_CC3E 0x0100
#define CCER_CC1E 0x01
// Flags for CR1
#define COUNTER_ENABLE (uint16_t) 0x01
// Flags for SR
#define READ_INPUT_CAPTURE1 (uint16_t) 0x02
#define SR_CLR_INPUT_CAPTURE ~((uint16_t) 0x2)
#define CH3_CC3IF 0x1 << 3
```

```
#define TIM_UIF 0x1
// Flags for DIER
#define CH3_CC_INTERRUPT_ENABLE 0x8
#define UPDATE_INTERRUPT_ENABLE 0x1

void initTIM3AsPWMMode(void) {
    uint16_t * reg_pointer;
    /* 90 MHz / (250 * (11+1)) = 30 kHz */
    uint16_t prescale = 11;
    uint16_t auto_reload = 250;
    /* Enable clock */
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM3, ENABLE);
    /* Map to B0 */
    initGpioB0AsAF2();
    /* Clear update event flag */
    reg_pointer = (uint16_t *) TIM3_SR_REG;
    *reg_pointer = 0;
    /* Upload prescale value */
    reg_pointer = (uint16_t*) TIM3_PRESCALE_REG;
    *reg_pointer = prescale;
    /* Set period value */
    reg_pointer = (uint16_t*) TIM3_AUTORELOAD_REG;
    *reg_pointer = auto_reload;
    /* Select PWM mode 1 for channel 3*/
    reg_pointer = (uint16_t*) TIM3_CAPTURE_COMPARE_MODE_2_REG;
    *reg_pointer = *reg_pointer | TIM_OC3M_PWMmode;
    /* Set channel 3 as output */
    *reg_pointer = *reg_pointer & ~((uint16_t)0x2);
    /* Set value to compare to (sets duty cycle) */
    reg_pointer = (uint16_t *) TIM3_COMP3_REG;
    *reg_pointer = 0; // init at 0% (will be set by input)
    /* Enable preload register */
    reg_pointer = (uint16_t*) TIM3_CAPTURE_COMPARE_MODE_2_REG;
    *reg_pointer = *reg_pointer | TIM_OC3PE;
    /* Enable TIM3 channel 3 */
    reg_pointer = (uint16_t *) TIM3_CAPTURE_COMP_ENABLE_REG;
    *reg_pointer = *reg_pointer | CCER_CC3E;
    /* Enable timer subsystem */
    reg_pointer = (uint16_t *) TIM3_CR1_REG;
    *reg_pointer = *reg_pointer | COUNTER_ENABLE;
```

```
}

void setPWMDutyCycle(float percent) {
    uint16_t auto_reload = 250; // autoreload value
    uint16_t* reg_pointer;
    // If percent is negative, set motor to spin cw
    if (percent < 0) {
        clearGpioB(1);
        percent = percent * -1; // Absolute value of percent
    } else { // If percent positive, motor should spin ccw
        setGpioB(1);
    }
    // If GPIOB is set (ccw) then PWM wave needs to be inverted
    // ie. when wave low, motor is on
    if (checkGpioB(1) > 0) { // GPIOB is set (ccw)
        percent = 1 - percent;
    }
    // Set PWM to new percent
    reg_pointer = (uint16_t *) TIM3_COMP3_REG;
    *reg_pointer = (uint16_t)(auto_reload * percent);
}
```

Hardware_stm_timer3.h

```
#ifndef __HARDWARE_STM_TIMER3_H_
#define __HARDWARE_STM_TIMER3_H_

#ifdef __cplusplus
 extern "C" {
#endif

/* Includes
------------------------------------------------------------------*/
#include "main.h"

/* Macros for
Everyone----------------------------------------------------------*/

/*Function
definitions-------------------------------------------------------------*/
void initTIM3AsPWMMode(void);
```

```
void setPWMDutyCycle(float percent);

#ifdef __cplusplus
}
#endif

#endif
```

Hardware_stm_timer4.c

```
#define TIM4_BASE_ADDR (uint32_t) 0x40000800
#define TIM4_CR1_REG TIM4_BASE_ADDR
#define TIM4_SR_REG (TIM4_BASE_ADDR + 0x10)
#define TIM4_CAPTURE_COMPARE_MODE_1_REG (TIM4_BASE_ADDR + 0x18)
#define TIM4_CAPTURE_COMPARE_MODE_2_REG (TIM4_BASE_ADDR + 0x1C)
#define TIM4_PRESCALE_REG (TIM4_BASE_ADDR + 0x28)
#define TIM4_AUTORELOAD_REG (TIM4_BASE_ADDR + 0x2C)
#define TIM4_COMP4_REG (TIM4_BASE_ADDR + 0x40)
#define TIM4_COMP3_REG (TIM4_BASE_ADDR + 0x3C)
#define TIM4_COMP2_REG (TIM4_BASE_ADDR + 0x38)
#define TIM4_CAPTURE_COMP_ENABLE_REG (TIM4_BASE_ADDR + 0x20)
#define TIM4_INTERRUPT_ENABLE_REG (TIM4_BASE_ADDR + 0x0C)

// Flags for CR1
#define COUNTER_ENABLE (uint16_t) 0x01
// Flags for TIM4_CCMR registers
#define TIM_OC4PE (0x1 << 11)
#define TIM_OC3PE (0x08)
#define TIM_OC2PE (0x1 << 11)
// Flags for CCER
#define CCER_CC4E (0x1 << 12)
#define CCER_CC3E 0x0100
#define CCER_CC2E (0x1 << 4)
// Flags for DIER
#define CH4_CC_INTERRUPT_ENABLE (uint16_t)(0x1 << 4)
#define CH3_CC_INTERRUPT_ENABLE (uint16_t)(0x1 << 3)
#define CH2_CC_INTERRUPT_ENABLE (uint16_t)(0x1 << 2)
#define UPDATE_INTERRUPT_ENABLE 0x1
// Flags for SR
#define CH4_CC4IF (0x1 << 4)
#define CH3_CC3IF (0x1 << 3)
```

```
#define CH2_CC2IF (0x1 << 2)
#define TIM_UIF 0x1

// Inits channels 2-4 of TIM4
void initTIM4ToInterrupt(void) {
    uint16_t * reg_pointer;
    // Values for 1 Hz (1s) timer
    uint16_t prescale = 1373;
    uint16_t auto_reload = 0xFFFF;

    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM4, ENABLE);
    enableNVIC_Timer4();

    /* Clear update event flag */
     reg_pointer = (uint16_t *) TIM4_SR_REG;
    *reg_pointer = 0;
    /* Upload prescale value */
    reg_pointer = (uint16_t*) TIM4_PRESCALE_REG;
    *reg_pointer = prescale;
    /* Set period value */
    reg_pointer = (uint16_t*) TIM4_AUTORELOAD_REG;
    *reg_pointer = auto_reload;
    /* Set value to compare to (sets duty cycle) */
    reg_pointer = (uint16_t *) TIM4_COMP4_REG;
    *reg_pointer = auto_reload/2;
    reg_pointer = (uint16_t *) TIM4_COMP3_REG;
    *reg_pointer = auto_reload/2;
    reg_pointer = (uint16_t *) TIM4_COMP2_REG;
    *reg_pointer = auto_reload/2;
    /* Enable preload reg */
    reg_pointer = (uint16_t*) TIM4_CAPTURE_COMPARE_MODE_2_REG;
    *reg_pointer = *reg_pointer | TIM_OC4PE | TIM_OC3PE;
    reg_pointer = (uint16_t*) TIM4_CAPTURE_COMPARE_MODE_1_REG;
    *reg_pointer = *reg_pointer | TIM_OC2PE;
    /* Enable TIM4 channel 4 */
    reg_pointer = (uint16_t *) TIM4_CAPTURE_COMP_ENABLE_REG;
    *reg_pointer = *reg_pointer | CCER_CC4E | CCER_CC3E | CCER_CC2E;
    /* Enable timer subsystem */
    reg_pointer = (uint16_t *) TIM4_CR1_REG;
    *reg_pointer = *reg_pointer | COUNTER_ENABLE;
```

```
}

// Channel 4 is the event timer.
// If no correct action is taken with the time limit
// the game moves on to the next event.
static int event_count = 0;
void startEventTimer(void) {
    event_count = 0; // Reset count when timer started
    uint16_t * reg_pointer;
    reg_pointer = (uint16_t *) TIM4_INTERRUPT_ENABLE_REG;
    *reg_pointer = *reg_pointer | CH4_CC_INTERRUPT_ENABLE;
}

void stopEventTimer(void) {
    uint16_t * reg_pointer;
    reg_pointer = (uint16_t *) TIM4_INTERRUPT_ENABLE_REG;
    *reg_pointer = *reg_pointer & (~CH4_CC_INTERRUPT_ENABLE);
}

// Channel 3 is the game timer. The game runs for 30s.
void startGameTimer(void) {
    uint16_t * reg_pointer;
    reg_pointer = (uint16_t *) TIM4_INTERRUPT_ENABLE_REG;
    *reg_pointer = *reg_pointer | CH3_CC_INTERRUPT_ENABLE;
}

void stopGameTimer(void) {
    uint16_t * reg_pointer;
    reg_pointer = (uint16_t *) TIM4_INTERRUPT_ENABLE_REG;
    *reg_pointer = *reg_pointer & (~CH3_CC_INTERRUPT_ENABLE);
}

// Channel 2 is the no action timer.
// If the games goes 30s without user input, it resets
// to welcoming mode.
int no_action_count = 0;
void resetNoActionTimer(void) {
    no_action_count = 0; // Reset when there is a user action
}
```

```
void startNoActionTimer(void) {
    uint16_t * reg_pointer;
    no_action_count = 0; // Reset when timer started
    reg_pointer = (uint16_t *) TIM4_INTERRUPT_ENABLE_REG;
    *reg_pointer = *reg_pointer | CH2_CC_INTERRUPT_ENABLE;
}

void stopNoActionTimer(void) {
    uint16_t * reg_pointer;
    reg_pointer = (uint16_t *) TIM4_INTERRUPT_ENABLE_REG;
    *reg_pointer = *reg_pointer & (~CH2_CC_INTERRUPT_ENABLE);
}

void TIM4_IRQHandler(void) {
    uint16_t * reg_pointer_sr;
    uint16_t * reg_pointer_dier;
    reg_pointer_sr = (uint16_t *) TIM4_SR_REG;
    reg_pointer_dier = (uint16_t *) TIM4_INTERRUPT_ENABLE_REG;

    static int game_count = 0; // For total game runtime
    // Check if event timer fired
    if (( (*reg_pointer_sr & CH4_CC4IF) > 0) && ( (*reg_pointer_dier &
CH4_CC_INTERRUPT_ENABLE) > 0)) {
        // Clear interrupt
        *reg_pointer_sr = ~((uint16_t)CH4_CC4IF);
        event_count = event_count + 1;
        if (event_count == 6) { // 6 seconds til move on to next event
            event_count = 0;
            addEvent(NEXT_EVENT_TIMEOUT); // Main state machine will move
to next event
            stopEventTimer(); // Stop timer
        }
    }
    // Game timer fired
    if (( (*reg_pointer_sr & CH3_CC3IF) > 0) && ( (*reg_pointer_dier &
CH3_CC_INTERRUPT_ENABLE) > 0)) {
        // Clear interrupt
        *reg_pointer_sr = ~((uint16_t)CH3_CC3IF);
        game_count = game_count + 1;
```

```
        // If we are at 15 or 5 seconds left, print to serial monitor so
that Processing can play time left audio
        if (game_count == 15) {
            debugprint(15);
        } else if (game_count == 25) {
            debugprint(25);
        } else if (game_count == 30) { // If game is at 30s, stop game and
add event to stop everything else
            addEvent(GAME_TIMEOUT);
            game_count = 0;
            stopGameTimer();
        }
    }
    // No action timer fired
    if (( (*reg_pointer_sr & CH2_CC2IF) > 0) && ( (*reg_pointer_dier &
CH2_CC_INTERRUPT_ENABLE) > 0)) {
        *reg_pointer_sr = ~((uint16_t)CH2_CC2IF);
        no_action_count = no_action_count + 1;
        // 31s is to add a slight offset from the main game timer to
guarentee that even is no action is taken
        // by the user after starting the game, the game over timer still
finishes first
        if(no_action_count == 31) {
            no_action_count = 0;
            addEvent(NO_ACTION_TIMEOUT);
            stopNoActionTimer();
        }
    }
    // Check if overflow event fired
    if (( (*reg_pointer_sr & TIM_UIF) > 0) && ( (*reg_pointer_dier &
UPDATE_INTERRUPT_ENABLE) > 0)) {
        // Clear interrupt
        *reg_pointer_sr = ~((uint16_t) TIM_UIF);
    }
    *reg_pointer_sr = 0; // clear any remaining flags (overflower, etc.)
}
```

Hardware_stm_timer4.h

```
#ifndef __HARDWARE_STM_TIMER4_H_
#define __HARDWARE_STM_TIMER4_H_
```

```
#ifdef __cplusplus
 extern "C" {
#endif

/* Includes
----------------------------------------------------------------------*/
#include "main.h"

/*Function
definitions-----------------------------------------------------------------*/
void initTIM4ToInterrupt(void);
void startEventTimer(void);
void stopEventTimer(void);
void startGameTimer(void);
void stopGameTimer(void);
void startNoActionTimer(void);
void stopNoActionTimer(void);
void resetNoActionTimer(void);

#ifdef __cplusplus
}
#endif

#endif
```

Motor_control.c

```
#define FULL_ROTATION (48*4.4) // 48 counts and 4.4 gear ratio
static float goal_position = 0; // Where we want motor to be
static float current_position = 0; // Where motor actually is (according
to encoder)

// Sets the vibration motor pin high and starts timer to turn off
void pulseVibrationMotors(void) {
    setGpioB(6);
    startVibrationTimer();
}

// Wrapper function to turn off vibration pin for readability
void stopVibrationMotors(void) {
```

```
    clearGpioB(6);
}

void initMotors(void) {
    // Vibration motor control pin
    initGpioBAsOutput(6);
    // Hourglass motor control pins
    initGpioBAsOutput(1); // controls motor direction
    initTIM3AsPWMMode(); // B0 is mapped to TIM3 for PWM
    // C8 and C9 are encoder inputs, enabled to interrupt
    initGpioCAsInput(8);
    initGpioCAsInput(9);
    // If we had used the PID controller, these would be uncommented
    // enableEXTI8OnPortC();
    // enableEXTI9OnPortC();
}

// Convert angle to counts and set goal_position
void rotateMotor(float angle) {
    float change = (angle / 360) * FULL_ROTATION;
    goal_position = goal_position + change;
}

// Used instead of PID controller, number found with trial and error
void flipMotor(void) {
    setPWMDutyCycle(0.274);
    startMotorTimer();
}

// We did not actually use this function, but it did work fairly well
without the ADC/DMA
void PIDController() {
    static float e_prev = 0; // Used for derivative control
    static int prev_percent = 0;
    static float net_e = 0;
    // Values after a ton of tuning, still definitely not perfect
    float kp = 1.5;
    float ki = 0; // Had trouble with Ki, even a tiny value created a lot
of oscillations
    float kd = 1; //0.2; // 0.4
```

```
    // Position calculated by encoder A & B (C9 and C8 interrupts)
    current_position = (float) getCount();
    // Calculate error
    float e = goal_position - current_position;
    // Calculate net error accumulation
    // Implement anti-windup (only add error term if not at full effort)
    if (prev_percent < 1 && prev_percent > -1) {
        net_e = net_e + e;
    }
    // Calculate error differential
    float e_diff = e - e_prev;
    e_prev = e;
    float pid = kp * e + ki * net_e + kd * e_diff;

    // 220 determined experientially so that percent usually between
    // 1 and -1
    float percent = pid/220;
    if(percent > 1) { // Make sure percent is actually a percent (between
0 and 1)
        percent = 1;
    } else if (percent < -1) {
        percent = -1;
    }
    // Turn motor on again
    prev_percent = percent;
    setPWMDutyCycle(percent);
}
```

Motor_control.h

```
#ifndef __MOTOR_H_
#define __MOTOR_H_

#ifdef __cplusplus
 extern "C" {
#endif

/* Includes
------------------------------------------------------------------*/
#include "main.h"
```

```
/*Function
definitions-------------------------------------------------------------*/
void rotateMotor(float angle);
void initMotors(void);
void flipMotor(void);
void pulseVibrationMotors(void);
void stopVibrationMotors(void);
void PIDController(void);

#ifdef __cplusplus
}
#endif

#endif
```

Math.c

```
const uint32_t a = 16807;
const uint32_t m = 2147483647; // 2^31 - 1

// Tested by generating 10,000 random numbers and got
// 0, 1, 2, and 3 2512, 2473, 2529, and 2486 times respectively

// Uses specific LCG, multiplicative congruential generator
// Pseudo random, everytime microcontroller is restarted the sequence will
repeat
int random(int x) {
    static uint32_t seed = 1;
    seed = (seed * a) % m; // LCG formula
    // Higher bits "more" random so shift these bits down
    // and mod to scale answer.
    return ((seed>>22) % x);
}
```

Math.h

```
#ifndef __MATH_
#define __MATH_

#ifdef __cplusplus
 extern "C" {
#endif
```

```
/*Function
definitions-----------------------------------------------------------*/
int random(int x);

#ifdef __cplusplus
}
#endif

#endif
```

Inputs.c

```
void initAllInputs(void) {
    // Init all input pins
    initGpioDAsInput(4); // D4 is the motion sensor
    initGpioDAsInput(5); // D5 is the stomp limit switch
    initGpioDAsInput(6); // D6 is the shake switch
    initGpioDAsInput(7); // D7 is the pull limit switch

    // Enable all input pins to interrupt
    enableEXTIOnPortD_4_7(4);
    enableEXTIOnPortD_4_7(5);
    enableEXTIOnPortD_4_7(6);
    enableEXTIOnPortD_4_7(7);
}

// These are all simple wrapper functions to improve readability of other
modules
uint32_t checkPull(void) {
    return checkGpioD(7);
}

uint32_t checkStomp(void) {
    return checkGpioD(5);
}

void restartStompEXTI(void) {
    restartEXTI(5);
}
```

```
void stopStompEXTI(void) {
    stopEXTI(5);
}

void restartPullEXTI(void) {
    restartEXTI(7);
}

void stopPullEXTI(void) {
    stopEXTI(7);
}
```

Inputs.h

```
#ifndef __INPUTS_H_
#define __INPUTS_H_

#ifdef __cplusplus
 extern "C" {
#endif

/* Includes
------------------------------------------------------------------*/
#include "inputs.c"

/*Function
definitions-------------------------------------------------------------*/
void initAllInputs(void);
uint32_t checkPull(void);
uint32_t checkStomp(void);
void restartStompEXTI(void);
void stopStompEXTI(void);
void restartPullEXTI(void);
void stopPullEXTI(void);

#ifdef __cplusplus
}
#endif

#endif
```

Led_array.c

```
#define ARRAY_SIZE 8 // Each shift register is connected to an array of 8
leds
// Min and max values are the range of the potentiometer in the controller
#define MAX_ADC_VAL 3000
#define MIN_ADC_VAL 500
#define NUM_LEVELS 17 // 16 LEDs plus level 0 when they are all off
#define STEP ((MAX_ADC_VAL-MIN_ADC_VAL) / NUM_LEVELS) // Number of ADC
counts per step


int led = 0; // Led level between 0 and 16
void initLEDs(void) {
    // Init ADC for led control input from analog pin F7
    initADC3_5();
    // Init clock pins for shift registers 1 and 2
    initGpioBAsOutput(8);
    initGpioBAsOutput(9);
    // Init reset pins for shift registers 1 and 2
    initGpioBAsOutput(15);
    initGpioBAsOutput(13);
    setGpioB(13);
    setGpioB(15);
    // After this delay, start the conversion
    startADC3Conversion();
}

// Clear then set the reset lines to clear all leds
void resetAllLEDs() {
    clearGpioB(13);
    clearGpioB(15);
    setGpioB(13);
    setGpioB(15);
    led = 0;
}

// Only clear the first array
void resetArray1() {
    clearGpioB(13);
    setGpioB(13);
```

```
}

// Only clear the second array
void resetArray2() {
    clearGpioB(15);
    setGpioB(15);
}

// Generates a clock wave by setting and clearing clock line
void clockWaveArray1() {
    setGpioB(8);
    clearGpioB(8);
}

void clockWaveArray2() {
    setGpioB(9);
    clearGpioB(9);
}

// Sets the LED after the currently lit level
void setNextLED() {
    if(led < 8) { // Next led is in first array
        clockWaveArray1();
        led = led + 1;
    } else if (led < 16) { // Next led is in second array
        clockWaveArray2();
        led = led + 1;
    }
}

// Clear the last currently lit led
void clearLastLED() {
    if (led <= 8) { // Last led is in first array
        led = led - 1;
        resetArray1(); // Clear all leds
        for (int i = 0; i < led; i ++) { // Set up to current level - 1
            clockWaveArray1();
        }

    } else if (led <= 16) { // Last led is in second array
```

```
        led = led - 1;
        resetArray2(); // Only clear the second array
        for (int i = ARRAY_SIZE; i < led; i ++) { // Set up to the current
level - 1
            clockWaveArray2();
        }
    }
}

// Light up to a certain LED (eg. up to 7 would light up first 7 LEDs)
void lightUpToLED(uint16_t upToLED) {
    resetAllLEDs();
    led = 0;
    for (int i = 0; i < upToLED; i ++) {
        setNextLED();
    }
}

// Twist State Machine (states OFF, WAIT FOR ALL OFF, WAIT FOR ALL ON)
// CheckTwist checks current potentiometer level, lights up correct LEDs,
and generates any events
void checkTwist(void) {
    static int prev_level = 0;
    uint16_t curr;
    int count = getADCData(); // Get current potentiometer value
    // Subtract out the min threshold unless below min (technically should
not happen)
    if (count > MIN_ADC_VAL) {
        curr = count - MIN_ADC_VAL;
    } else {
        curr = 0;
    }

    uint16_t level = (curr / STEP); // Level should be between 0 and 16

    if (prev_level != level) { // Only change if level change
        lightUpToLED(level); // Light up LEDs
    }

    // Generates events for the twist state machine
```

```
    if(level == 0) { // All the leds are off
       addEvent(TWIST_ALL_OFF);
    } else if (level == NUM_LEVELS - 1) { // All leds are on if level is
16
       addEvent(TWIST_ALL_ON);
    }
    prev_level = level; // Store current level
}
```

Led_array.h

```
#ifndef __LED_ARRAY_
#define __LED_ARRAY_

#ifdef __cplusplus
 extern "C" {
#endif

/* Includes
------------------------------------------------------------------*/
#include "main.h"

/*Function
definitions-----------------------------------------------------------*/
void initLEDs(void);
void resetAllLEDs(void);
void setNextLED(void);
void clearLastLED(void);
void lightUpToLED(uint16_t upToLED);
void checkTwist(void);

#ifdef __cplusplus
}
#endif

#endif
```

Events.c

```
/* Implementation for singly linked lists */
// Each block holds the value (event) and a pointer to the next block
typedef struct eventBlock_t {
```

```
    event_t event;
    struct eventBlock_t* next;
} eventBlock_t;

// List contains pointer to the first and last blocks in the list
// (so that we can easily access first and last) and the total length
typedef struct eventList_t {
    uint32_t length;
    eventBlock_t* first;
    eventBlock_t* last;
} eventList_t;

// List that holds all unserviced events
eventList_t list = {0};

// Returns the value of the first block in the list and deletes it from
the list
event_t removeEvent(void) {
    // Empty list, nothing to remove, so return NO_EVENT
    if (list.length == 0) {
        return NO_EVENT;
    }
    // Return the event at the front of the list
    event_t result = list.first->event;
    // Move the front of the list one block
    eventBlock_t* next = list.first->next;
    eventBlock_t* prev = list.first;
    list.first = next;
    // Free memory from the removed block
    free(prev);
    // Decrement the length
    list.length = list.length - 1;
    return result;
}

// Adds a block with the given event to the end of the list
void addEvent(event_t event) {
    // Create a new block
    eventBlock_t* newEventBlock = malloc(sizeof(eventBlock_t));
    // Set the value of event to be the given argument
```

```
    newEventBlock->event = event;
    // If the list is empty, set both first and last pointers to point to
the new block
    if (list.length == 0) {
        list.first = newEventBlock;
        list.last = newEventBlock;
    }
    // If the list is not empty, add block to the end
    else {
        // Set current last block to point to the new block
        list.last->next = newEventBlock;
        // Set the new block to be the last block
        list.last = list.last->next;
    }
    // Increment the lengths
    list.length = list.length + 1;
}

/* Service Interrupts Function */
// Looks at next event in list and sends to correct state machine
void serviceEventList(void) {
    event_t currEvent;
    // If the list is empty, add no event
    if (list.length == 0) {
        currEvent = NO_EVENT;
    }
    // Remove the first event from the event list if not empty
    else {
        currEvent = removeEvent();
    }
    switch(currEvent) {
        case NO_EVENT: // Call checkTwist() or PID controller in main
state machine
            mainStateMachine(NO_EVENT);
            break;
        // All input events (wave, pull, twist, shake, and stomp) are sent
to main state machine
        case WAVE:
            mainStateMachine(WAVE);
            break;
```

```
        case PULL:
            mainStateMachine(PULL);
            break;
        case TWIST:
            mainStateMachine(TWIST);
            break;
        case SHAKE:
            mainStateMachine(SHAKE);
            break;
        case STOMP:
            mainStateMachine(STOMP);
            break;
        // Pull and stomp timeouts and rising edges are sent to debouncing
state machine
        case PULL_TIMEOUT:
            debounceStateMachine(PULL_TIMEOUT);
            break;
        case STOMP_TIMEOUT:
            debounceStateMachine(STOMP_TIMEOUT);
            break;
        case PULL_RISING_EDGE:
            debounceStateMachine(PULL_RISING_EDGE);
            break;
        case STOMP_RISING_EDGE:
            debounceStateMachine(STOMP_RISING_EDGE);
            break;
        // Twist all off and all on events are used to determine a full
twist
        // and are sent to the twist state machine
        case TWIST_ALL_OFF:
            twistStateMachine(TWIST_ALL_OFF);
            break;
        case TWIST_ALL_ON:
            twistStateMachine(TWIST_ALL_ON);
            break;
        // Next event, no action, and game timeouts are all handled in the
main state machine.
        // Next event timeouts are also sent to debounce and twist state
machines so that
```

```
        // they can reset for the next event in case they were in the
middle of processing an event.
        case NEXT_EVENT_TIMEOUT:
            twistStateMachine(NEXT_EVENT_TIMEOUT);
            debounceStateMachine(NEXT_EVENT_TIMEOUT);
            mainStateMachine(NEXT_EVENT_TIMEOUT);
            break;
        case NO_ACTION_TIMEOUT:
            mainStateMachine(NO_ACTION_TIMEOUT);
            break;
        case GAME_TIMEOUT:
            mainStateMachine(GAME_TIMEOUT);
            break;
        default:
            break;
    }
}
```

Events.h

```
#ifndef __EVENTS_H_
#define __EVENTS_H_

#ifdef __cplusplus
 extern "C" {
#endif

/* Includes
------------------------------------------------------------------*/
#include "main.h"

/* Data structures
-----------------------------------------------------------*/
typedef enum {
    NO_EVENT,
    PULL,
    TWIST,
    SHAKE,
    STOMP,
    WAVE,
    GAME_TIMEOUT,
```

```
    NO_ACTION_TIMEOUT,
    NEXT_EVENT_TIMEOUT,
    STOMP_TIMEOUT,
    PULL_TIMEOUT,
    PULL_RISING_EDGE,
    STOMP_RISING_EDGE,
    TWIST_ALL_OFF,
    TWIST_ALL_ON
} event_t;

/*Function
definitions-----------------------------------------------------------------*/
void addEvent(event_t event);
event_t removeEvent(void);
void serviceEventList(void);

#ifdef __cplusplus
}
#endif

#endif
```

State_machine.c

```
// All possible states for the main state machine
typedef enum {
   WELCOME,
   WAIT_FOR_STOMP,
   WAIT_FOR_TWIST,
   WAIT_FOR_PULL,
   WAIT_FOR_SHAKE,
   GAME_OVER
} main_state_t;

// Variable that holds the current state, initialized to WELCOME
main_state_t current_state = WELCOME;

// Use random int generator to get int between 0 and 3 inclusive
// map int to one of 4 action states and print event to serial
// monitor so Processing can play correct prompt
main_state_t getRandomState(void) {
```

```
    int x = random(4); // X will be between 0 and 3
    startEventTimer(); // Timer until next event started
    switch(x) {
        case 0:
            printEvent(PULL);
            return WAIT_FOR_PULL;
        case 1:
            printEvent(TWIST);
            return WAIT_FOR_TWIST;
        case 2:
            printEvent(STOMP);
            return WAIT_FOR_STOMP;
        case 3:
            resetShake(); // Make sure shake count is 0
            printEvent(SHAKE);
            return WAIT_FOR_SHAKE;
        // Should never reach default case if random() works correctly,
have just in case
        default:
            // start stomp interrupt
            //restartEXTI(5);
            printEvent(STOMP);
            return WAIT_FOR_STOMP;
    }
}

// State machine that handles user actions and main game states
void mainStateMachine(event_t event) {
    static int score = 0;
    switch(event) {
        case WAVE:
            if(current_state == WELCOME || current_state == GAME_OVER) {
                // print wave to serial monitor, screen & audio handled by
Processing
                printEvent(WAVE);
                current_state = getRandomState();
                startGameTimer(); // start game 30s timer
                startNoActionTimer(); // start inaction timer
                flipMotor(); // flip hourglass
                score = 0; // Set score to 0
```

```
            }
        break;
        case STOMP:
            if(current_state == WAIT_FOR_STOMP) {
                stopEventTimer(); // turn off event timer
                resetNoActionTimer(); // User action so no action timer
restarted
                current_state = getRandomState();
                // pulse vibration motors
                pulseVibrationMotors();
                // Increment score
                score = score + 1;
            }
        break;
        case TWIST:
            if(current_state == WAIT_FOR_TWIST) {
                stopEventTimer(); // turn off event timer
                resetNoActionTimer();
                current_state = getRandomState();
                resetAllLEDs(); // Make sure all LEDs are off
                // pulse vibration motors
                pulseVibrationMotors();
                score = score + 1;
            }
        break;
        case PULL:
            if(current_state == WAIT_FOR_PULL) {
                stopEventTimer(); // turn off event timer
                resetNoActionTimer();
                current_state = getRandomState();
                // pulse vibration motors
                pulseVibrationMotors();
                score = score + 1;
            }
        break;
        case SHAKE:
            if(current_state == WAIT_FOR_SHAKE) {
                stopEventTimer(); // turn off event timer
                resetNoActionTimer();
                current_state = getRandomState();
```

```
                // pulse vibration motors
                pulseVibrationMotors();
                score = score + 1;
            }
        break;
        case NEXT_EVENT_TIMEOUT:
            // User did not get action in time, choose a new state
            if(current_state != WELCOME && current_state != GAME_OVER) {
                printEvent(NEXT_EVENT_TIMEOUT);
                resetAllLEDs(); // If previous event was a twist, may need
to turn off leds
                current_state = getRandomState();
            }
        break;
        case GAME_TIMEOUT:
            current_state = GAME_OVER;
            resetAllLEDs(); // If final event was twist, may need to turn
off leds
            // Print game over and final score to serial monitor
            printEvent(GAME_TIMEOUT);
            printInt(score);
        break;
        case NO_ACTION_TIMEOUT:
            current_state = WELCOME; // Goes to welcoming state at no
action
            // print no action to serial monitor
            printEvent(NO_ACTION_TIMEOUT);
        break;
        case NO_EVENT:
            if(current_state == TWIST) {
                checkTwist();
            }
            // Control for hourglass motor goes here if doing PID
            // Also tried calling this in main loop, but still did not
work
            // PIDController();
        break;
        default:
        break;
    }
```

```
}
```

State_machine.h

```
#ifndef __STATE_MACHINE_H_
#define __STATE_MACHINE_H_

#ifdef __cplusplus
 extern "C" {
#endif

/* Includes
------------------------------------------------------------------*/
#include "main.h"
#include "events.h"


/*Function
definitions-------------------------------------------------------------*/
void mainStateMachine(event_t new_event);

#ifdef __cplusplus
}
#endif

#endif
```

State_machine_debounce.c

```
// Possible states for debounce state machine
typedef enum {
    WAIT_FOR_STOMP,
    WAIT_FOR_PULL,
    OFF
} debounce_state_t;

// Variable for current state, initialized to OFF
debounce_state_t current_debounce_state = OFF;

// Debounces both stomp and pull limit switches
// On a rising edges it starts timer, and on a timeout checks for a valid
event
```

```
// I did try splitting this into 2 different state machines, but that did
not fix bugs
void debounceStateMachine(event_t newEvent) {
    static int count = 0;
    switch(newEvent) {
        // If we have moved on from an event without finishing
        // reset state machine for next event.
        case NEXT_EVENT_TIMEOUT:
            current_debounce_state = OFF;
            restartPullEXTI();
            restartStompEXTI();
            break;
        case STOMP_RISING_EDGE:
            if (current_debounce_state == OFF) {
                // Start timer and wait for timeout to confirm if stomp
valid
                current_debounce_state = WAIT_FOR_STOMP;
                startStompDebounce();
            }
        break;
        case PULL_RISING_EDGE:
            if (current_debounce_state == OFF) {
                // Start timer and wait for timeout to confirm if pull
valid
                current_debounce_state = WAIT_FOR_PULL;
                startPullDebounce();
            }
        break;
        case PULL_TIMEOUT:
            if (current_debounce_state == WAIT_FOR_PULL) {
                if (checkPull() > 0) {
                    // If valid pull, then add a pull event to list
                    addEvent(PULL);
                }
                restartPullEXTI(); // Restart the interrupt for future
rising edges
                current_debounce_state = OFF; // Change state to off
            }
        break;
        case STOMP_TIMEOUT:
```

```
            if (current_debounce_state == WAIT_FOR_STOMP) {
                if (checkStomp() > 0) {
                    // If valid stomp, then add stomp event to list
                    addEvent(STOMP);
                }
                restartStompEXTI(); // Restart the interrupt for future
rising edges
                current_debounce_state = OFF; // Change state to off
            }
        break;
        default:
        break;
    }
}
```

State_machine_debounce.h

```
#ifndef __STATE_MACHINE_BUTTON_H_
#define __STATE_MACHINE_BUTTON_H_

#ifdef __cplusplus
 extern "C" {
#endif

/*-------Includes
------------------------------------------------------------------*/
#include "state_machines.h"

/*-------Function
definitions-----------------------------------------------------*/
void debounceStateMachine(event_t newEvent);

#ifdef __cplusplus
}
#endif

#endif
```

State_machine_twist.c

```
// Possible states for twist machine
typedef enum {
```

```
    WAIT_FOR_EMPTY1,
    WAIT_FOR_FULL,
    WAIT_FOR_EMPTY2
} twist_state_t;

// Variable for current state, initialized to waiting for first empty
twist_state_t current_twist_state = WAIT_FOR_EMPTY1;

// The goal of the twist state machine is to produce valid TWIST events
// A valid twist events happens when the user starts with all LEDs off
// turns them all on by fully twisting the potentiometer and then
// turns them all off again.
void twistStateMachine(event_t event) {
    switch(event) {
        case TWIST_ALL_OFF: // If we get an all off event, check if we are
waiting for first or second all off
            if(current_twist_state == WAIT_FOR_EMPTY1) {
                current_twist_state = WAIT_FOR_FULL;
            } else if(current_twist_state == WAIT_FOR_EMPTY2) {
                current_twist_state = WAIT_FOR_EMPTY1;
                addEvent(TWIST);
            }
            break;
        case TWIST_ALL_ON: // If we are waiting for an all on, change
state
            if(current_twist_state == WAIT_FOR_FULL) {
                current_twist_state = WAIT_FOR_EMPTY2;
            }
            break;
        case NEXT_EVENT_TIMEOUT: // In an event timeout, reset the machine
for the next time a twist event happens
            current_twist_state = WAIT_FOR_EMPTY1; // in case a twist
event timed out, reset
        default:
            break;
    }
}
```

State_machine_twist.h

```
#ifndef __STATE_MACHINE_TWIST_H_
```

```
#define __STATE_MACHINE_TWIST_H_

#ifdef __cplusplus
 extern "C" {
#endif

/* Includes
------------------------------------------------------------------*/
#include "main.h"
#include "events.h"

/*Function
definitions-------------------------------------------------------------*/
void twistStateMachine(event_t new_event);

#ifdef __cplusplus
}
#endif

#endif
```

Debug_mort.cpp

```
void debugprint(int32_t number)
{
    printf("Total count %d\n", number);
}

void debugprintHelloWorld( void )
{
    printf("I'm aliiiiiivveeee!!!\n");
}

// Prints events to serial monitor for communication with Processing
void printEvent(event_t event) {
    switch(event) {
        case WAVE:
            printf("Wave\n");
            break;
        case PULL:
            printf("Pull\n");
```

```
            break;
        case SHAKE:
            printf("Shake\n");
            break;
        case STOMP:
            printf("Stomp\n");
            break;
        case TWIST:
            printf("Twist\n");
            break;
        case GAME_TIMEOUT:
            printf("Game timeout\n");
            break;
        case NO_ACTION_TIMEOUT:
            printf("No action timeout\n");
            break;
        case NEXT_EVENT_TIMEOUT:
            printf("Nope, next!\n");
            break;
        default:
            break;
    }
}
```

Debug_mort.h

```
#ifndef __DEBUG_MORT_H_
#define __DEBUG_MORT_H_

#ifdef __cplusplus
 extern "C" {
#endif

/* Includes
------------------------------------------------------------------*/
#include "main.h"
#include "events.h"

/*Function
definitions-------------------------------------------------------------*/
void debugprint(int32_t number);
```

```
void debugprintHelloWorld( void );
void printEvent(event_t event);

#ifdef __cplusplus
}
#endif

#endif
```

8. Lessons Learned

For some general lessons learned:

- For any purchasing emails, be sure to double check the BOM and purchasing list up until it is sent to check for any last minute changes by any group members (avoid missing any crucial parts to buy before it is too late to purchase)

We also have hardware and software lessons that we have learned and would put more time and consideration into if we were to do this again.

8.1 Hardware

Use a much lower infill for printing, assuming that each iteration won't be the final.

While prototyping the controller, we used more infill than necessary (40%) towards the end, as we assumed those prints to be the final version. However, there were issues with these prints, ranging from hardware changes and out of date features being printed by mistake to printer malfunctions that required reprints. Near the end, we barely had enough filament to print everything that was required.

For smaller gears, use a larger module so gears don't slip.

While designing the gear train for the twist handle and potentiometer, we hadn't taken into account the size of these gears and how this might cause issues. Our first iteration had a few problems such as tolerancing, but the largest issue was the potential for the teeth to slip. The redesign used a much larger module so that slip would no longer be a concern.

Make electronics accessible for assembly and hardware debugging.

Once our hardware was finalized, we noticed that wiring and soldering connections became very difficult due to the hardware layout and organization. There were many tight spaces and corners, and reduced visibility for certain areas. This made electronics assembly more difficult than necessary and hardware debugging both time-consuming and error-prone.

Insulate and solder whatever wires you can.

Like aforementioned, it is difficult to look inside the enclosures to debug hardware. It becomes important to solder connection points so that they don't become loose and to insulate any exposed wires. This way, it is less likely that we consider loose or accidental contact as a reason for why the system was not working as intended.

When designing enclosures, consider how electronics will fit, overlap, and interact with the enclosed space.

When designing the main box, we wanted a hinged panel so that we can insert a computer at will to become the display. Naturally, the top panel became the hinged opening. However, once we started putting in all the electronics, we realized that the wiring and motors blocked the laptop from its intended location.

When designing enclosures or mounts, consider any points of wiring that needs to be plugged in.

For the main box, while we accounted for the power cables and connection wires, we forgot to include holes for the laptop itself (the USB cable that connects to the microcontroller) and also the laptop's power button (which kept bumping into the wall panels).

We also had a tested laser cut print of our main box enclosure, but we did not use it to the fullest potential and used it to organize all of our electronics and get a feel for how that felt before we laser cutted with the intended material.

Consider all of the work that has to be done in order to implement a feature

For our LED array we decided to use 16 LEDs. Designing and making the circuit and holder for the LEDs was simple enough, but we failed to account for the fact that we would have to solder and organize 32 long wires within the main box to make this feature work.

8.2 Software

Leave plenty of time for software debugging

Due to incompletion of hardware, we were unable to test many of our software features until much later on. This led to an exorbitant amount of time being spent last minute in order to pinpoint and fix software issues. This problem could have been avoided with better hardware planning, timeline discussion, and overall time management.

Plan out pins, timers, and interrupts before for better organization

Planning out pins, timers, and interrupts helps organize your code a lot better and having it all written down ahead of time makes it easier to structure and debug code.

Appendix - Electronics Datasheets

STM32 pinout with PWM timers and more info
Pololu 25d Gearmotor
L293B Motor Driver
CD4014BE Shift Register
LM324 Operational Amplifier
Logic Level Shifter, 4-Channel, Bidirectional
LabKit Diode 14N001
Vibration Motor | Datasheet
Motion Sensor Website | Datasheet for Motion Sensor
Motion Sensor | Adafruit datasheet